NestSelf authentication and authorization: quick guide

The ten-minute version: what the auth platform does, the decisions behind it and where to look. The detailed walkthrough is `implementation.md`; the full specification with every requirement is `docs/tasks/auth/auth-strategy.md`.

What it does

NestSelf serves five populations from one platform: employees of client companies, their HR and managers, independent experts, NestSelf staff and external suppliers. Each person has exactly one identity for life, may belong to several employers over time, and sees only what their current role allows. Authentication answers "who is this"; authorization answers "what may they do right now, in this organization".

The decisions

Decision	What we chose	Why
Identity provider	Amazon Cognito, one user pool per environment in Frankfurt (eu-central-1)	Email one-time codes, authenticator apps and passkeys without building them; free at pilot scale; EU data residency. Behind an interface so it can be replaced.
Sign-in methods	Employees use an email code. Experts and staff use a password plus an authenticator app. Employers can bring their own SAML or OIDC sign-in (enterprise SSO).	Removes passwords for the largest population; no SMS costs in Serbia and Hungary.
Web sessions	The browser gets an opaque cookie; the API keeps Cognito tokens encrypted in Redis (Backend-for-Frontend)	Tokens never reach JavaScript; revoking a session is deleting a key; CSRF handled with a double-submit token.
Mobile	The app talks to Cognito directly (PKCE) and calls the API with a bearer token	The standard native pattern; tokens live in the device's secure store.
Authorization	Hand-built grants evaluated by a pure `authorize()` function	Delegation, spending ceilings, effective-access previews and a 150-case golden test table are native; tokens carry no roles, so access changes apply within seconds.
Organization structure	A tree of org units with a closure table	Cheap ancestor queries for "manager of this team and everything under it".
Database access	Drizzle ORM with PostgreSQL row-level security	Tenant isolation lives in the database: a query without an organization filter returns nothing. Partial indexes, checks and views are part of the schema.
Audit	Append-only, partitioned `audit_events` with retention tiers	Every consequential action is recorded, in the same transaction as the change; nothing sensitive (tokens, codes, message bodies, raw IPs) ever is.
Email	One mailer for what the API sends itself (invitations, its own codes, the operations alert), through SES in AWS, sent only after the change that justifies it has committed	A rolled-back change never leaves a mail behind; without SES nothing is sent and nothing secret is logged.

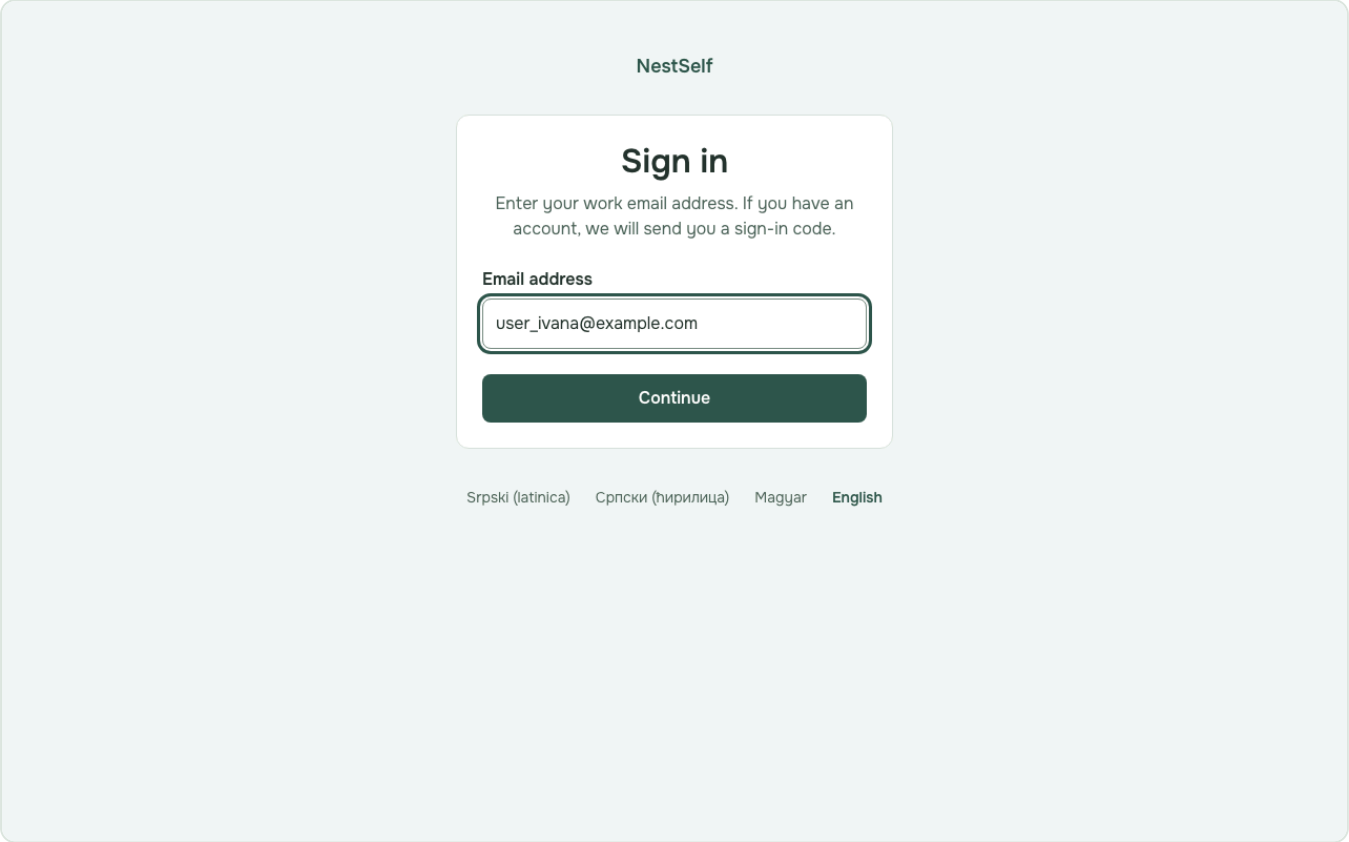
Decision	What we chose	Why
Background work	BullMQ on Redis, run by a separate worker process	Reconciliation (expired grants, stale mobile token families, audit partitions, expired sessions, expired emergency access, expiring SSO certificates) is audited like any user action and never runs inside the API process.
Step-up	Sensitive actions need a sign-in or a fresh email code from the last 5 to 30 minutes, depending on the action's class	Granting access, ending memberships, reading confidential records and emergency access stay safe on an unattended screen. Every class uses an email code for now; the authenticator for the emergency class is an owner decision. SSO users get a code NestSelf mails to their work address, because Cognito cannot prove that the employer's sign-in asked them again.
Legacy roles	Removed from the database; every permission is a grant	One source of truth. `db:backfill-grants` converted the old role columns before migration `0005` dropped them.
Organization ownership	Exactly one `employer_owner` per organization: the organization's superadmin, who grants and revokes access and can hand the role to another member. NestSelf staff can change anything for an organization, including transferring its ownership	One accountable person per client, support never locked out. A second owner is refused with 409 `owner_exists`.
Enforcement	Every route declares `@Public`, `@Authenticated` or `@Authorize`; a CI test fails otherwise	Default deny cannot be forgotten.
Rollout	Cognito first, platform stacks later, mobile last	Local development stays local; the CDK app deploys only the auth stack unless asked for more.

How a request works

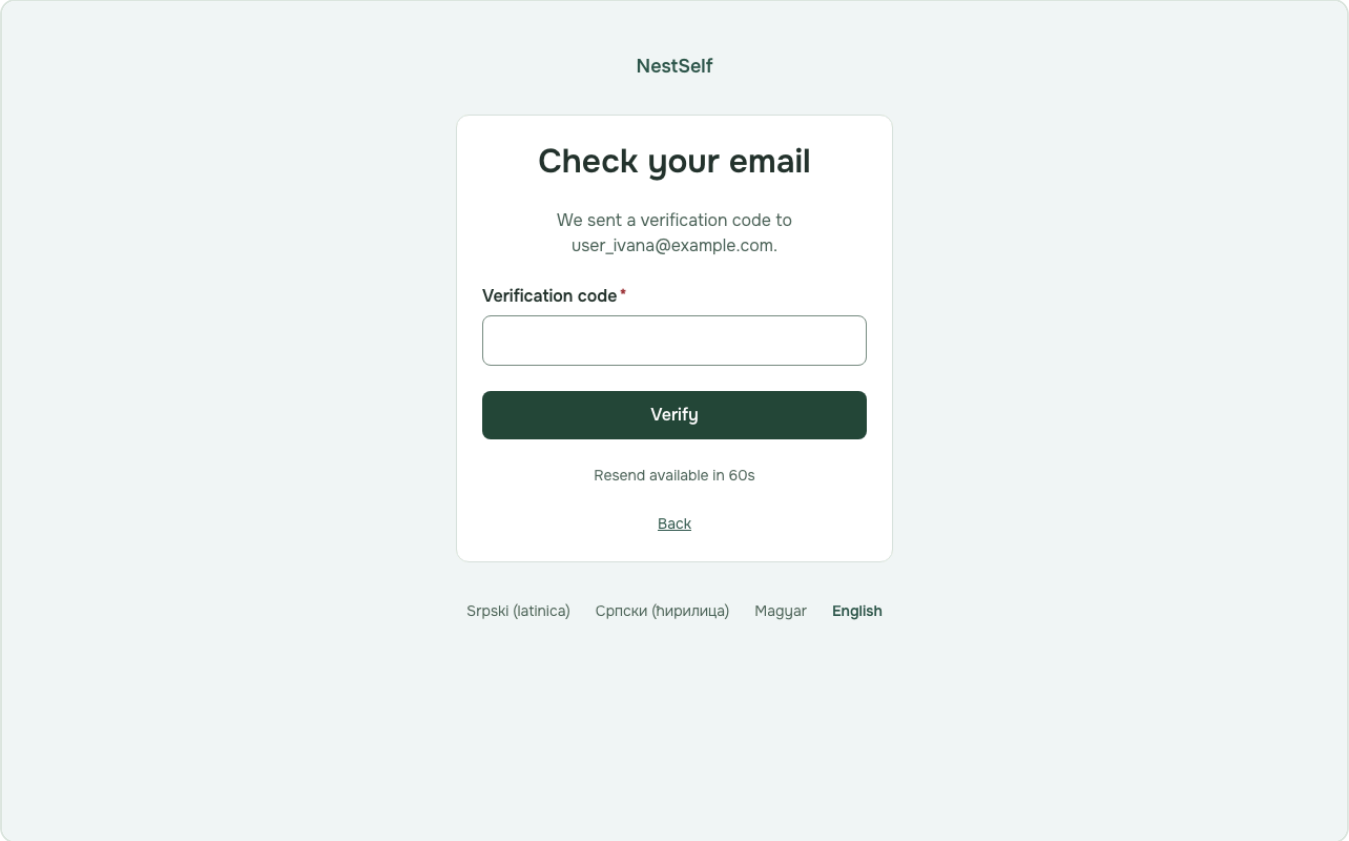
1. The browser sends the session cookie (or the app sends a bearer token).
2. `AuthenticationGuard` turns it into a `Principal`: who they are, their memberships, their active context.
3. `CsrfGuard` checks the CSRF header on mutating browser requests.
4. `AuthorizationGuard` reads the route's `@Authorize` (one action, or a list of which any one is enough), loads grants and assignments (cached in Redis for 30 to 60 seconds), and asks `authorize()`. Denied requests get 404 when the caller should not even know the record exists, 403 otherwise; a confidential record answers 404 for every refused read. For a sensitive action it also checks that the caller authenticated recently enough and answers 403 `step_up_required` if not.
5. A throttle counts requests per client address (and, on some routes, per session or per SSO connection), in Redis so every API task shares the budget.
6. The handler runs its writes inside an organization-scoped transaction; PostgreSQL row-level security blocks anything outside that organization.
7. Consequential actions are written to the audit log in that same transaction; access changes publish an invalidation that drops caches and disconnects affected WebSocket clients. Mails go out after the commit.

How sign-in works

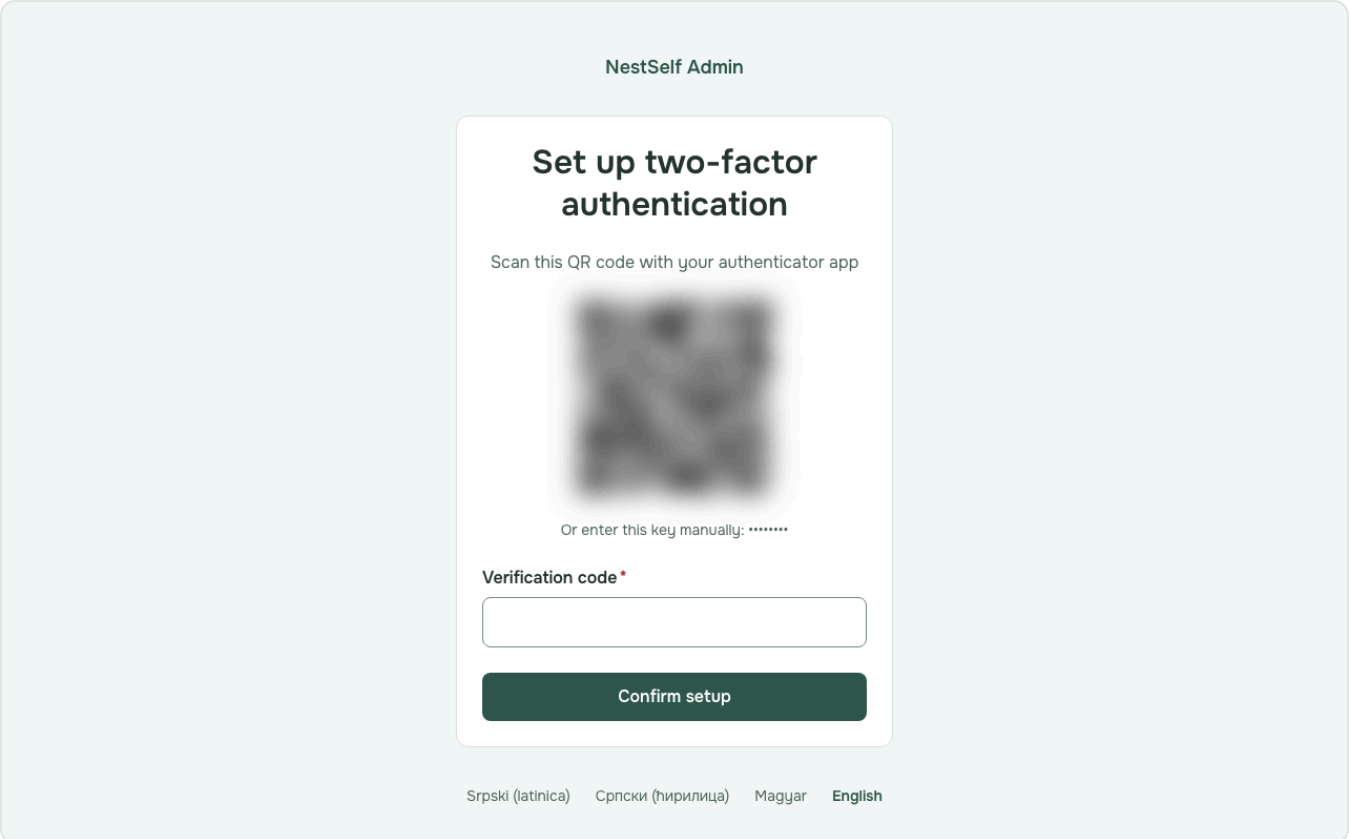
- **Employee**: enter email → receive an **eight digit** code (Cognito's length, not configurable) → enter it → session cookie. A wrong code can be retyped on the same challenge up to three times; after the third the user requests a new code, and five wrong codes in total lock the address for 15 minutes (`implementation.md`, section 6). The code email is written in the language of the sign-in screen.
- **Expert or staff**: enter email → password → authenticator code (first time: scan a QR code) → session cookie. Invited staff set a password first.
- **Employee of an employer with its own sign-in (SSO)**: enter the work email → "Redirecting to your organisation's sign-in..." → sign in at the employer → back in NestSelf. When the employer's OIDC sign-in does not confirm the address, NestSelf asks once for a code it mails to that address (the confirm-email step); the link carries an opaque handle, never the address.
- **Invitation**: HR invites by email (the mail goes out once the invitation is saved); the link previews the organization name and activates the membership on acceptance, then runs the matching sign-in, with the code email in the page's language.
- **Mobile**: email code directly against Cognito; the app refreshes tokens itself and uses short-lived tickets to open WebSockets. An SSO sign-in in the app is bound by the API first; an address the employer's OIDC sign-in did not confirm is confirmed once on the web.
- **When the account cannot be loaded** (the API answers with an error or not within 10 seconds), the web apps say so and offer Try again instead of signing the user out.



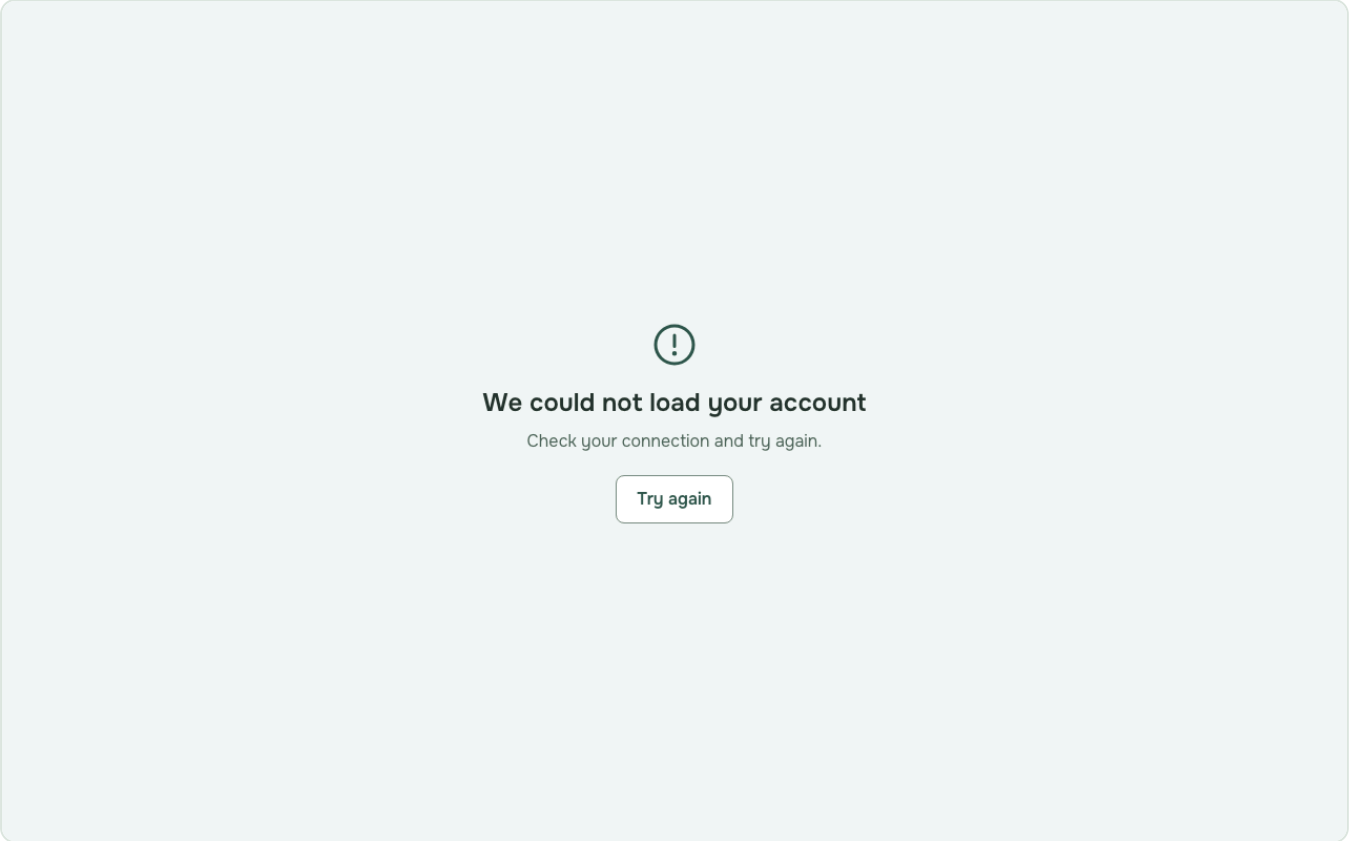
Workspace sign-in: one field first; the address decides the method.



Employees then type the code from the email.



Staff and experts enrol an authenticator on their first sign-in to the admin app. The key and the QR code carry the secret, so they are masked and blurred in the screenshot.



When the API cannot say whether there is a session, the page offers Try again; only a real "signed out" answer leads to the sign-in page.

Enterprise SSO

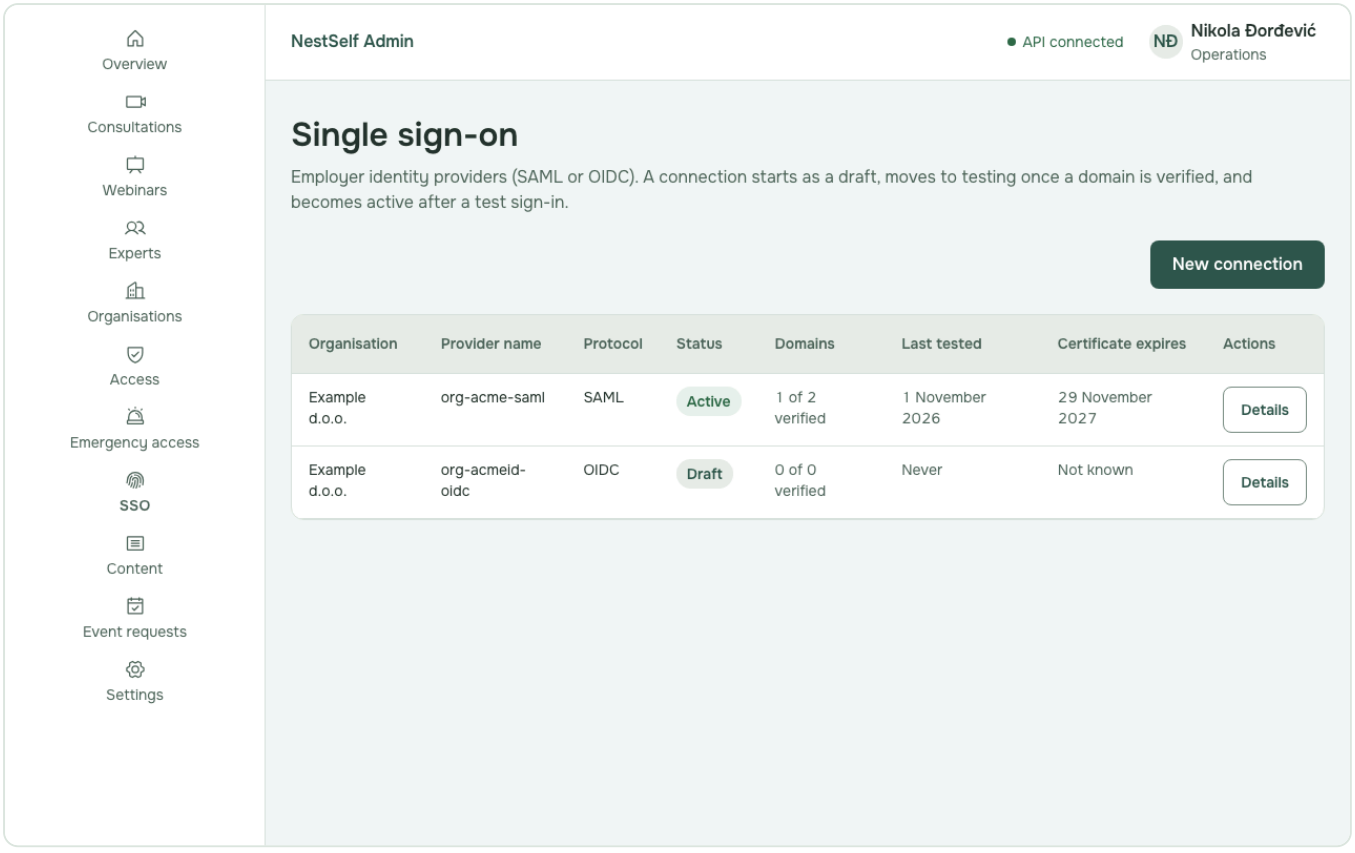
An employer can let its employees sign in through its own identity provider (SAML 2.0 or OIDC). Only sign-ins that start at NestSelf are accepted; the employer's portal cannot push people in.

What staff do (admin `/sso`, platform owners, every change after a recent sign-in or step-up):

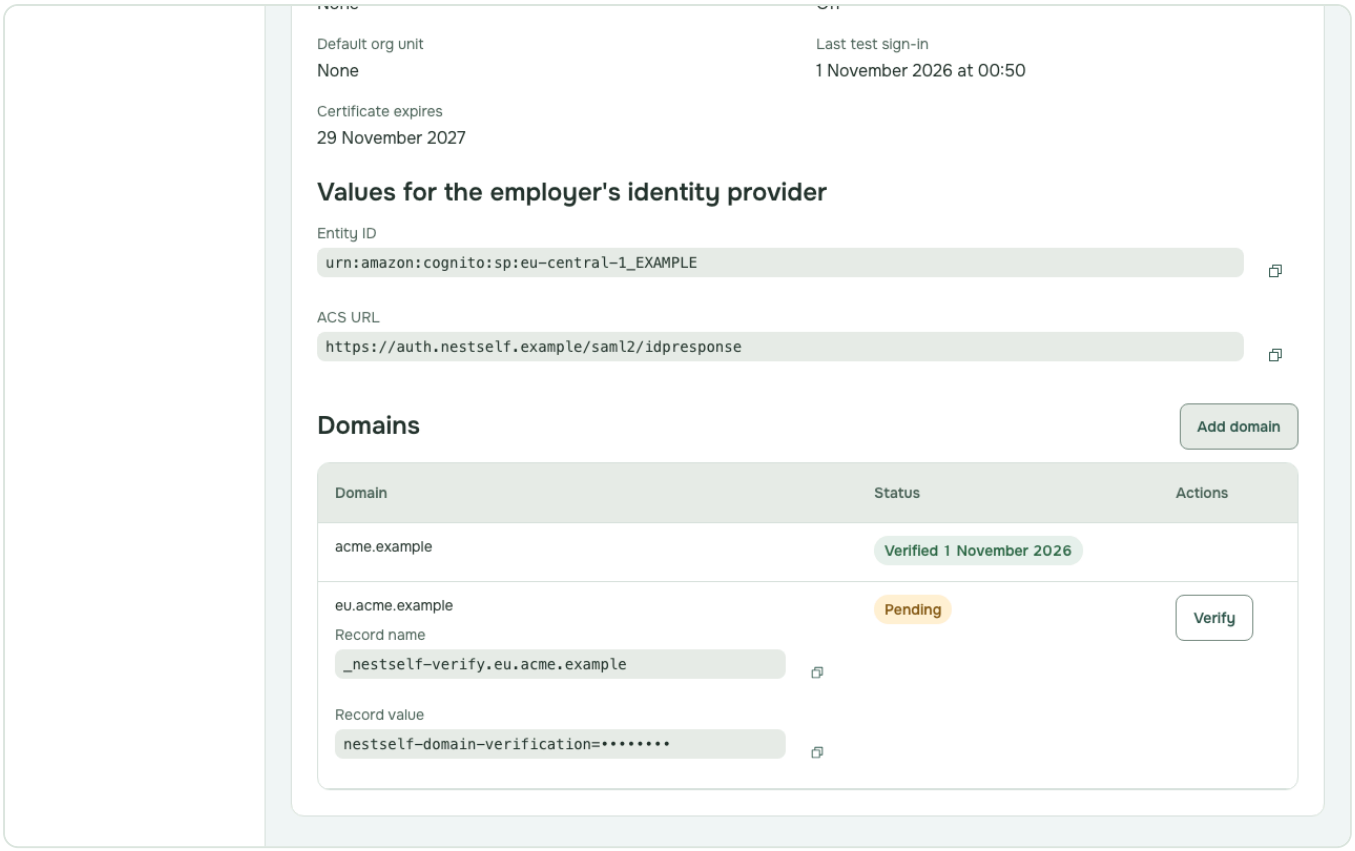
1. Create the connection for the organization (found with the search field): a slug (it builds the provider name, for example `org-acme-saml`) and the employer's metadata, or the OIDC issuer, client ID and secret. Metadata and issuer URLs must be public `https:` addresses. The secret goes straight to Cognito and is never shown or stored by NestSelf. A slug belongs to one organization: another organization cannot take it, for either protocol. For OIDC, "Require a verified email claim from the provider" stays on unless the employer confirms in writing that its email claim is the directory's verified address. The connection starts as a draft; a draft can still change its protocol.
2. Add the employer's email domains. Each gets a DNS TXT record (`_nestself-verify.<domain>`) that the employer publishes; then select Verify. Public mail domains such as `gmail.com` are refused.
3. Hand the employer the entity ID and ACS URL from the detail panel, then select Test sign-in. The page shows the link to open or copy; it works once and for 15 minutes, and an invited test person on a verified domain signs in with it (through the confirm-email step if the provider does not confirm the address). Reset to draft takes a connection in testing back to a draft and removes its provider.
4. Activate. From then on, discovery sends that employer's addresses to their sign-in. Suspend stops that until the connection is reactivated. Changing the provider settings of an active or suspended connection (metadata, issuer, client ID or secret) takes it back to testing: run a new test sign-in and activate it again.

What an employee sees: they type their work address on the normal sign-in screen, see "Redirecting to your organisation's sign-in…", sign in at their employer and land in NestSelf. If the employer's OIDC sign-in did not confirm the address, NestSelf first asks for the code it mailed there ("Confirm your email address"). NestSelf binds the account only for a pending invitation of that organization (or automatic accounts, if the employer turned them on), and only for an address on one of the connection's verified domains. Anyone else sees a neutral "Your organisation has not enabled your account yet" message. Too many attempts from one network show "wait a minute". The employer's sign-in counts as the second factor; for a sensitive action an SSO user gets a step-up code by email like everyone else.

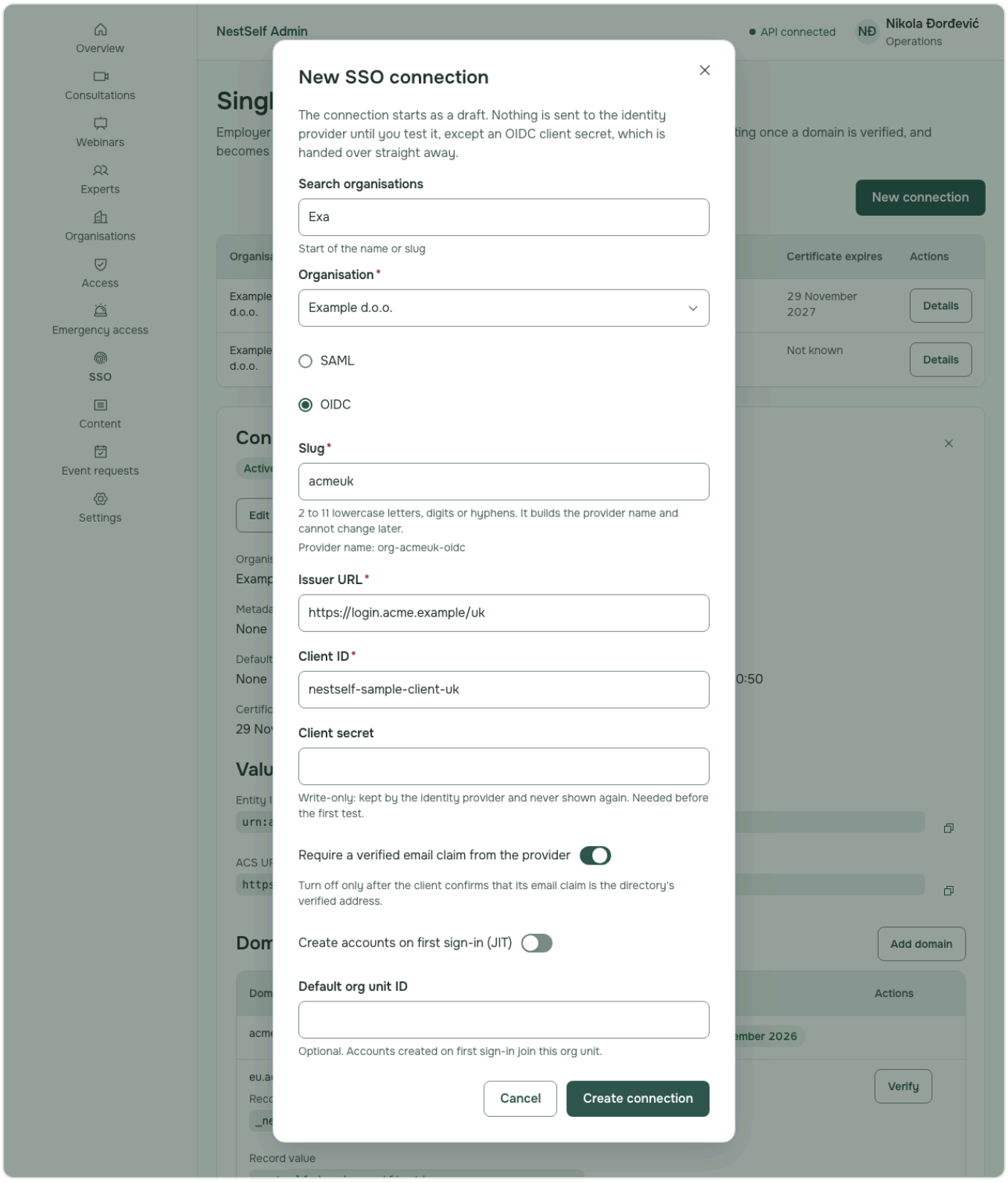
A nightly job reads each employer's signing certificate (from the metadata URL itself, when the connection has one) and warns when it expires within 30 days: an audit row and a log line every night, and a mail to the operations mailbox at most once a week.



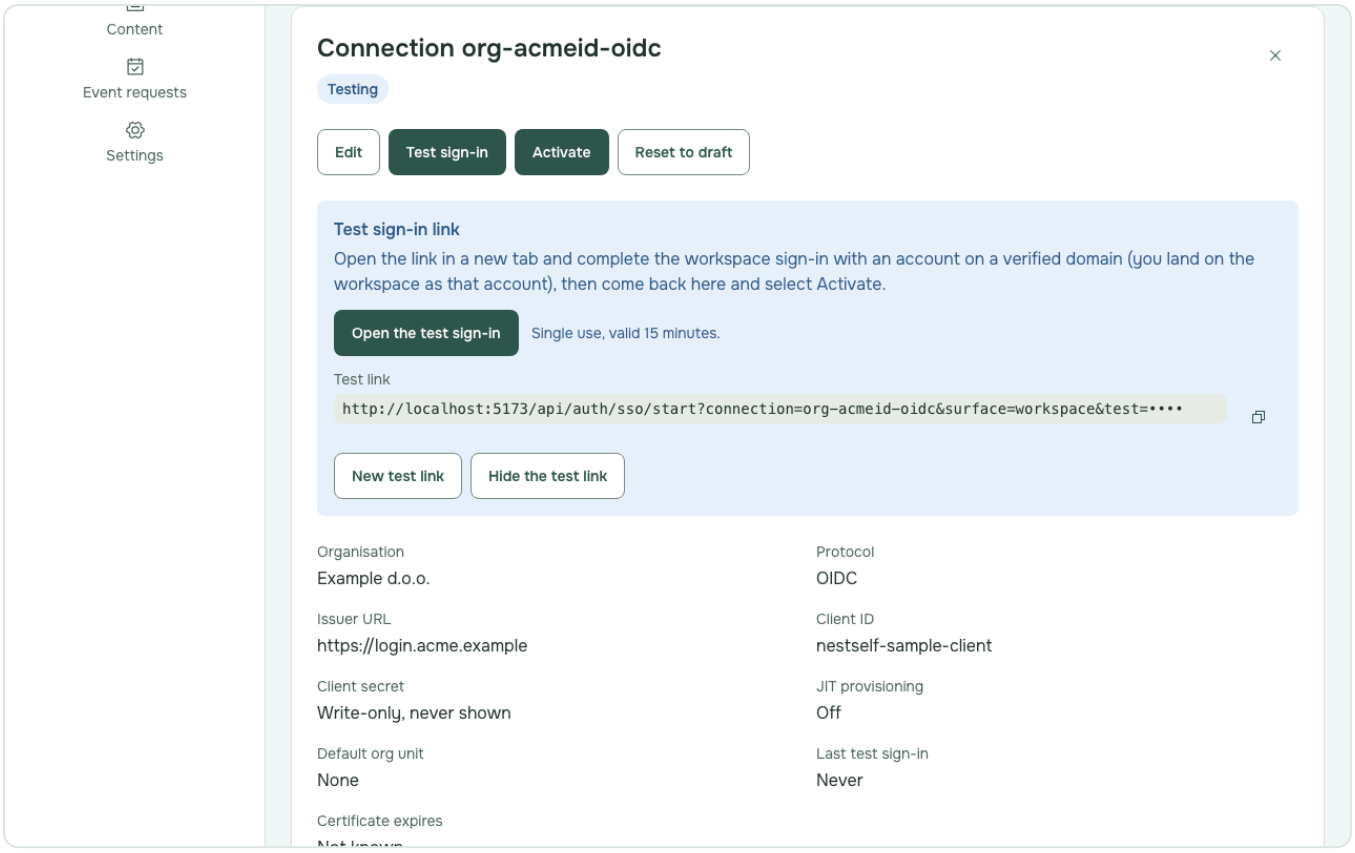
Staff: the SSO connections of the fictional sample organization, created through the stub API for the screenshot.



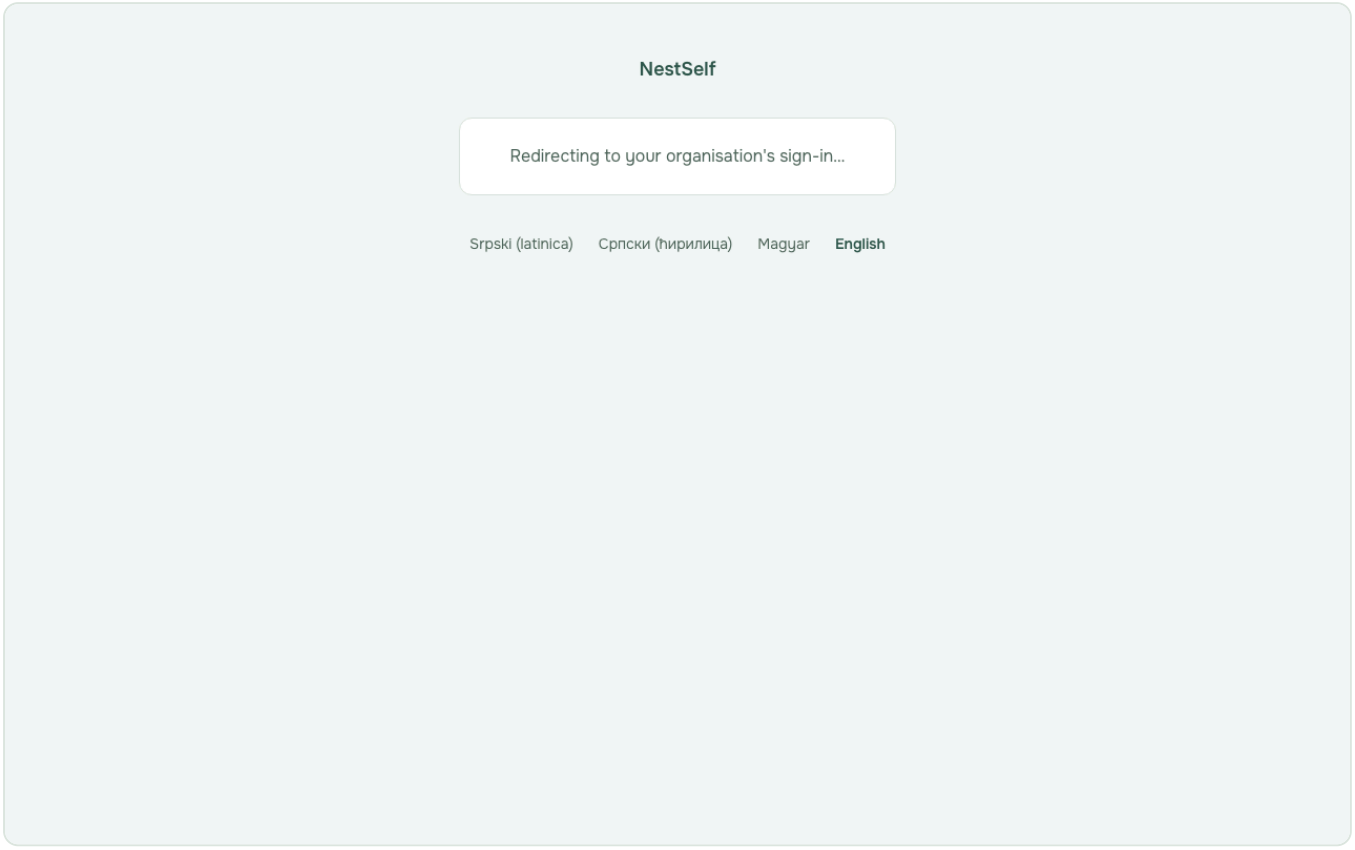
Staff: a connection's values for the employer and its domains; the pending one shows the TXT record to publish (its token masked in the screenshot).



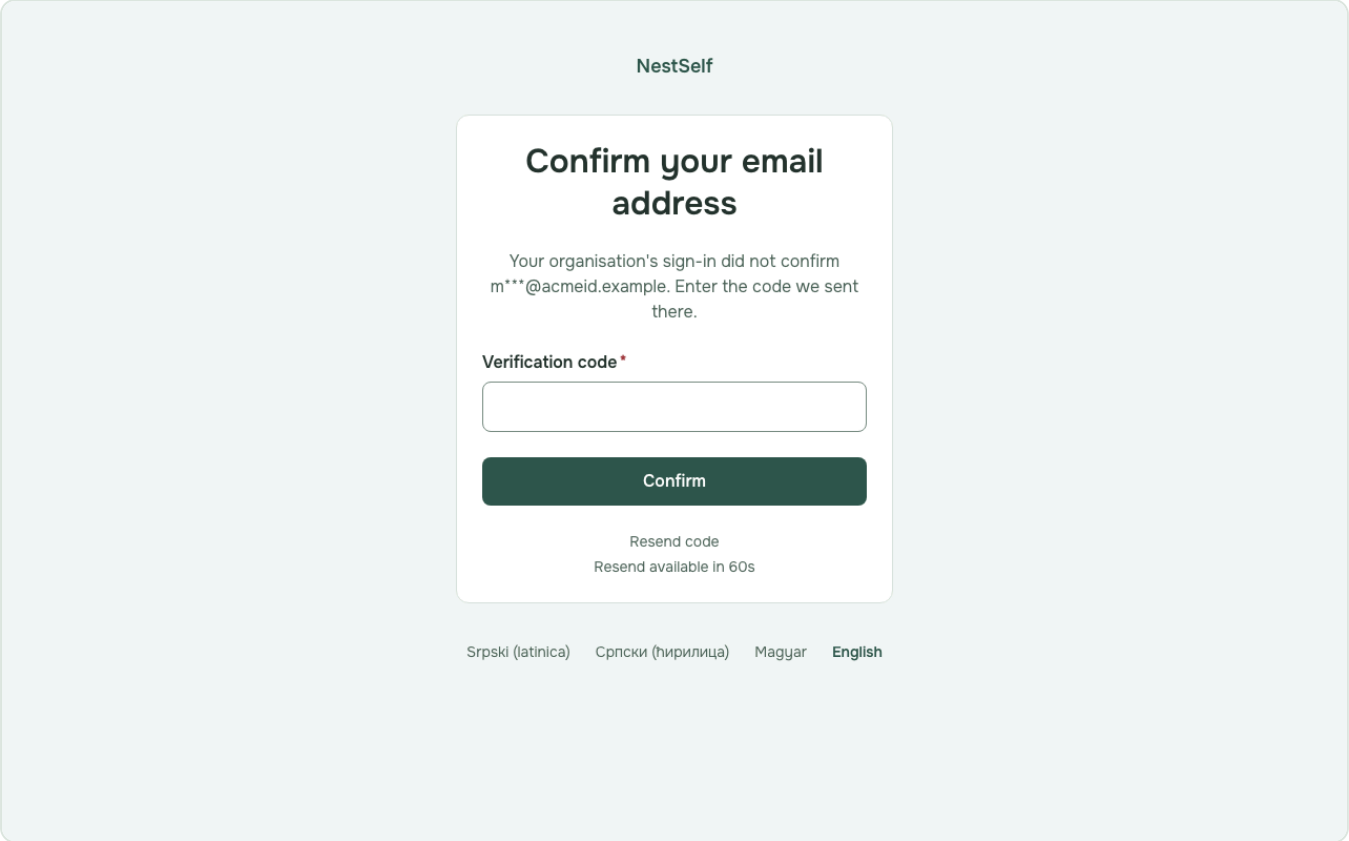
Staff: the create dialog with the organisation search and the OIDC claim switch, cancelled (a taller capture, so the whole form fits).



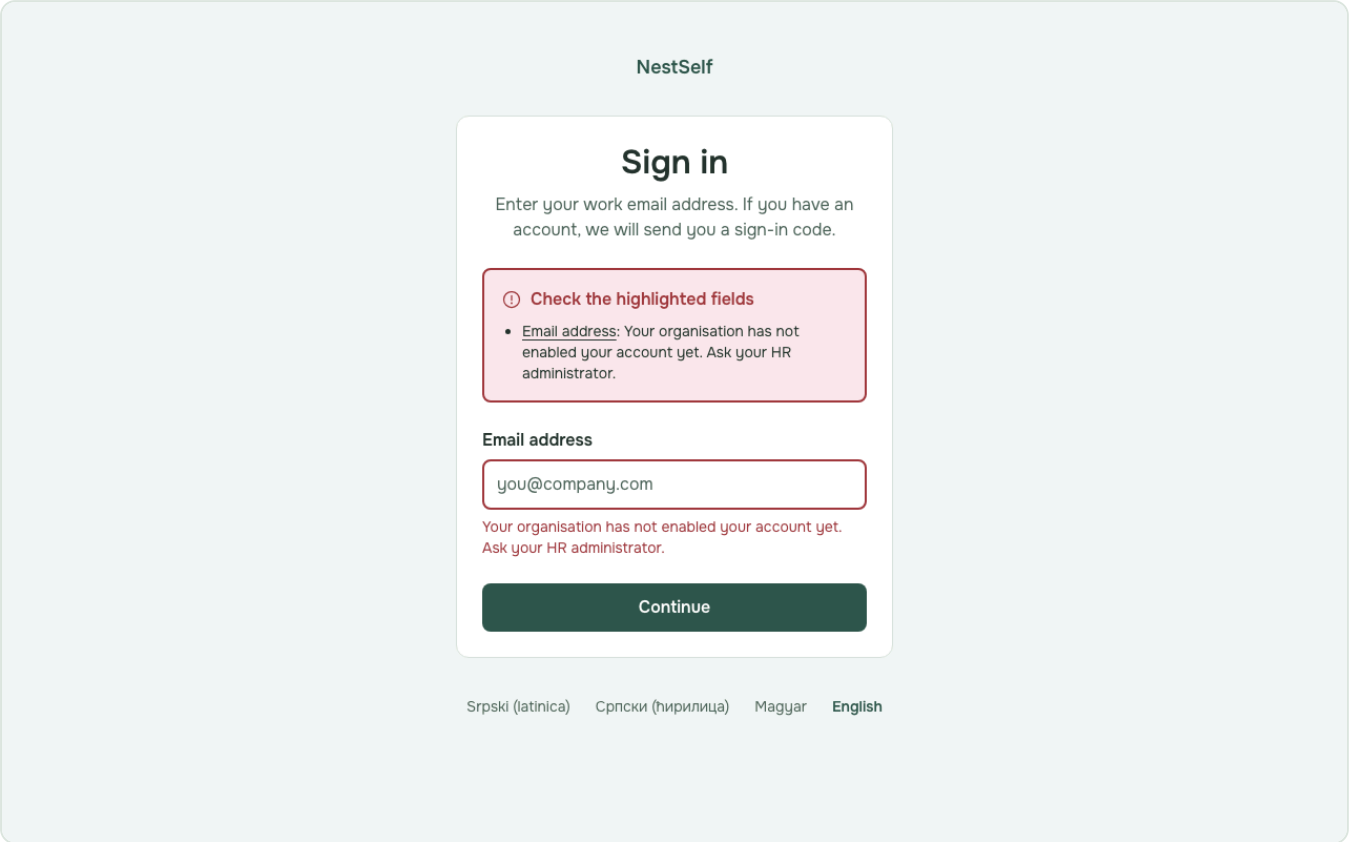
Staff: the single-use test link, to open in a new tab or copy (its nonce is masked in the screenshot).



Employee: discovery recognised the employer's domain. For the screenshot the browser was kept from leaving the page.



Employee: the employer's sign-in did not confirm the address, so NestSelf asks once for the code it mailed there.



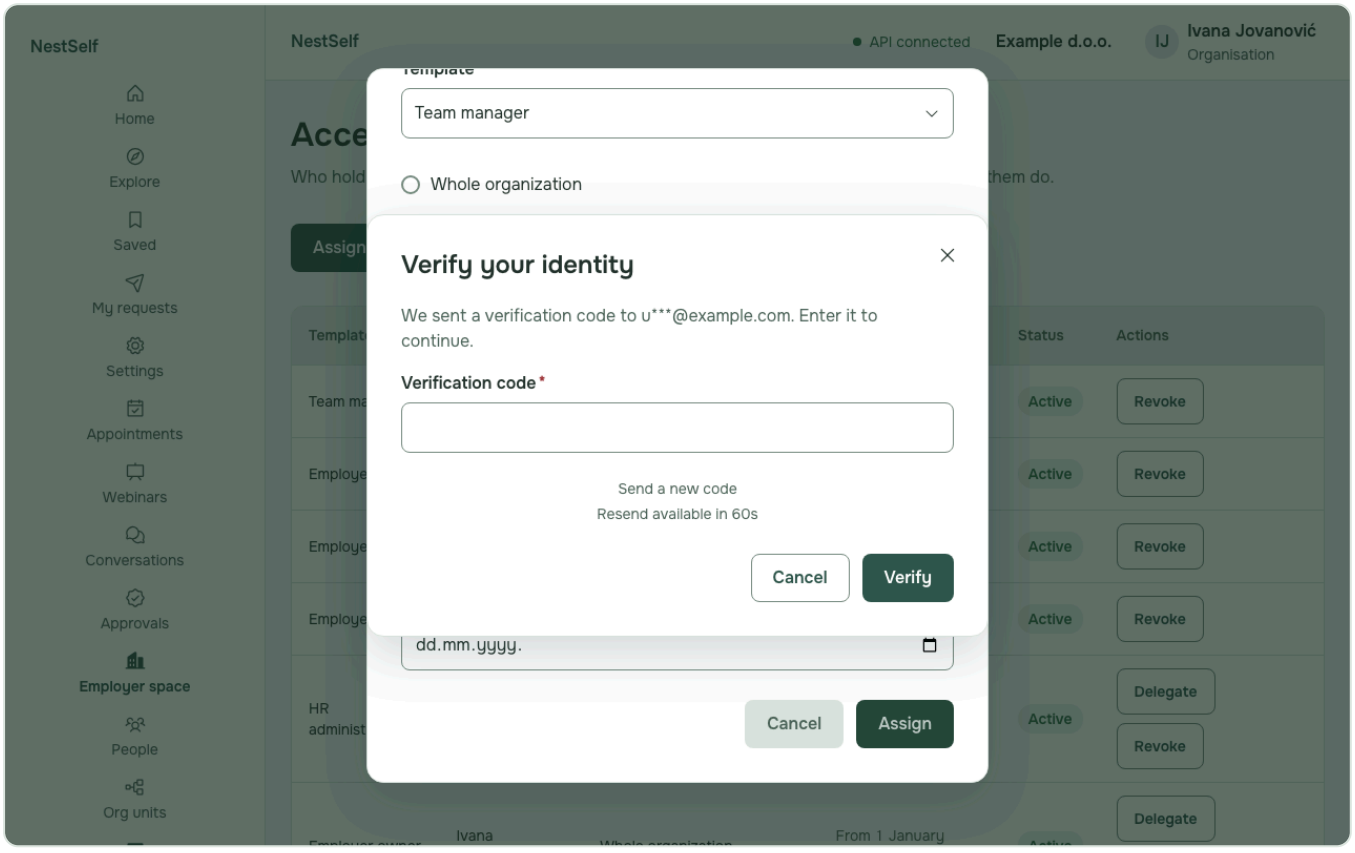
Employee: the employer signed the person in, but NestSelf had nothing to bind them to.

Everything is tested locally against a fake identity provider; the test against a real employer IdP waits for the pilot organisation, and the Cognito custom domain (`auth.<stage domain>`) must be deployed before the

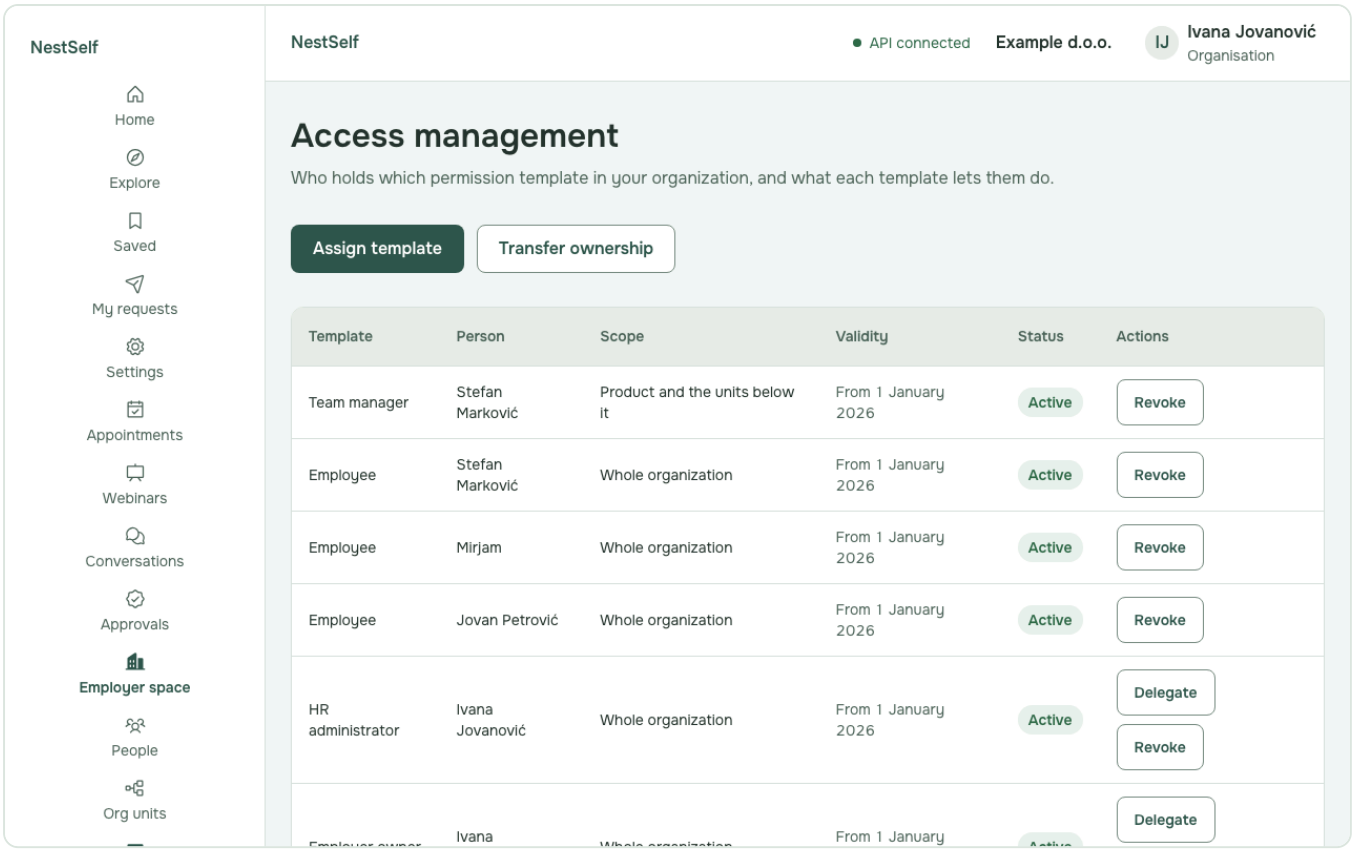
employer configures anything, because the values it registers derive from it (`implementation.md`, sections 17.2 and 22.13).

Step-up and the access screens

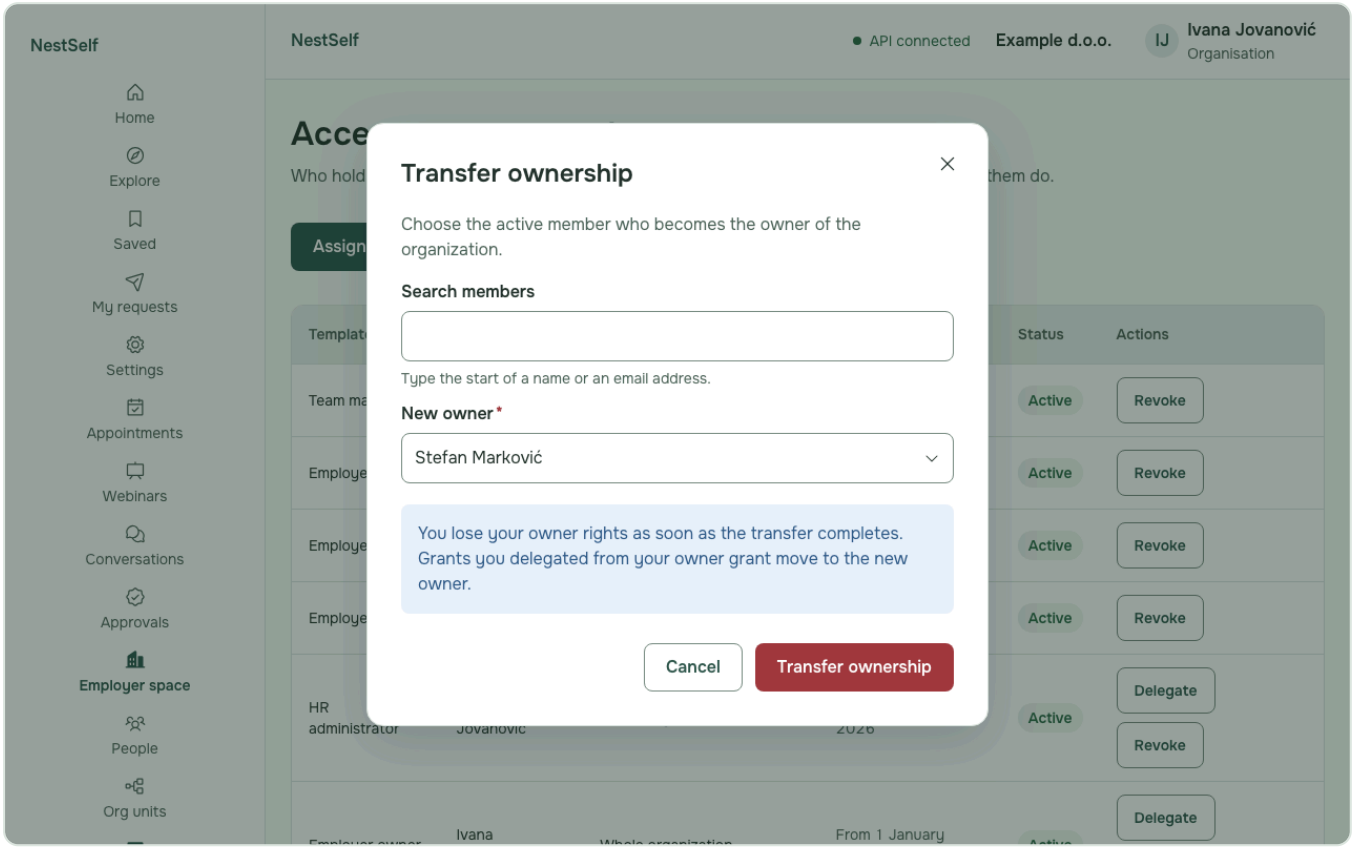
- **Step-up**: when the API answers `step_up_required`, the web apps open a dialog, email a fresh code and retry the action once the code is accepted. The dialog names the masked address the code went to; "Send a new code" works once a minute, with a countdown. SSO users get the code from NestSelf at their work address (Cognito cannot send them one); everyone else gets Cognito's code. Cancelling leaves the action undone without an error. Each code allows three tries, as in sign-in, and five wrong codes in 15 minutes lock the user's step-up for the rest of that window. An account with no address a code can go to is told to ask its administrator. Mobile has no step-up screen yet.
- **Workspace,** `/hr/access`: for the organization's owner, who needs no HR template for it (the sample Ivana is owner and HR administrator). Shows the organization's grants, previews any member's effective access, assigns and revokes templates, delegates part of one's own grant and transfers ownership to another active member. Every member choice has a "Search members" field, so members beyond the first 100 can be found.
- **Admin, organization Access tab and** `/access`: for platform owners. Assigns organization templates to an organization's members (organization-wide; org-unit grants are HR's job), transfers an organization's ownership (also when it has no owner yet), assigns staff templates, revokes grants and looks up any user's access per context.
- **Sample mode** shows the same screens over fixtures with every change disabled.



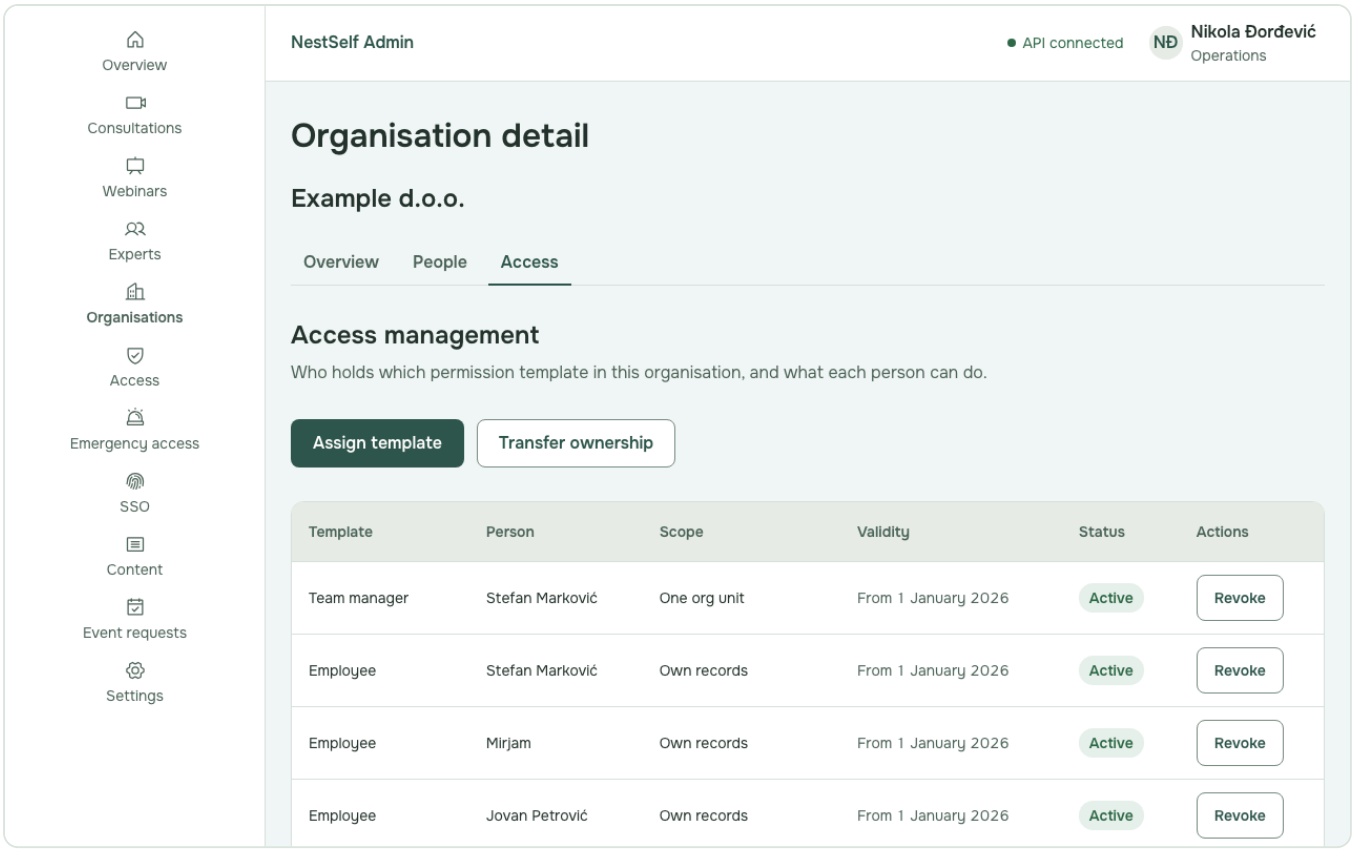
Step-up: a fresh email code before a sensitive change, here an assignment; the dialog names where the code went and when a new one can be sent.



Workspace `/hr/access` for the owner.



Transfer ownership: only the current owner sees it; the old owner's delegations move to the new owner.



The same organization seen by staff in the admin app.

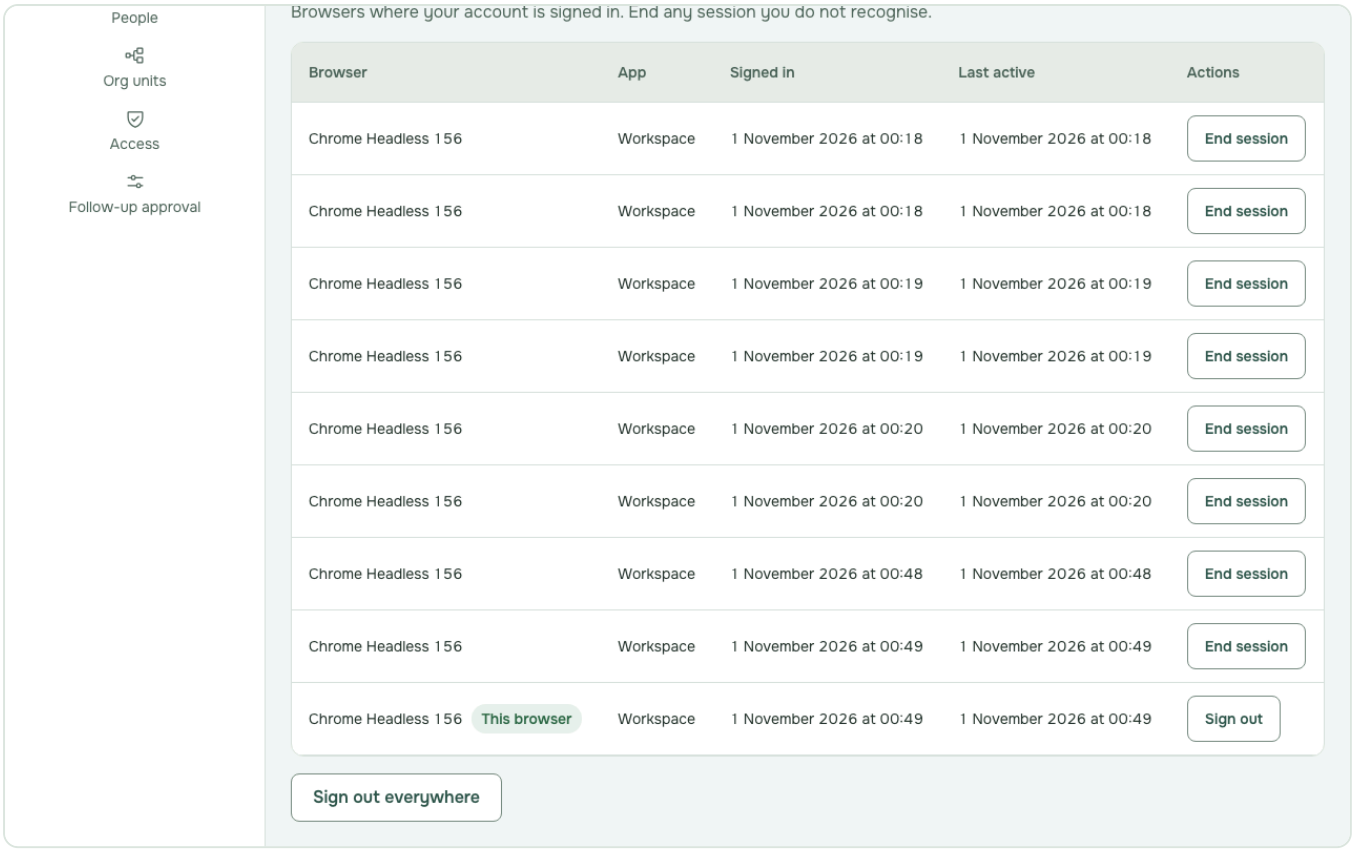
The screenshots are of the real screens against the stub API with the fictional sample people, with any key, token or nonce on screen masked; `pnpm docs:auth:screenshots` regenerates them (`implementation.md`, section 19).

What each role can do and where

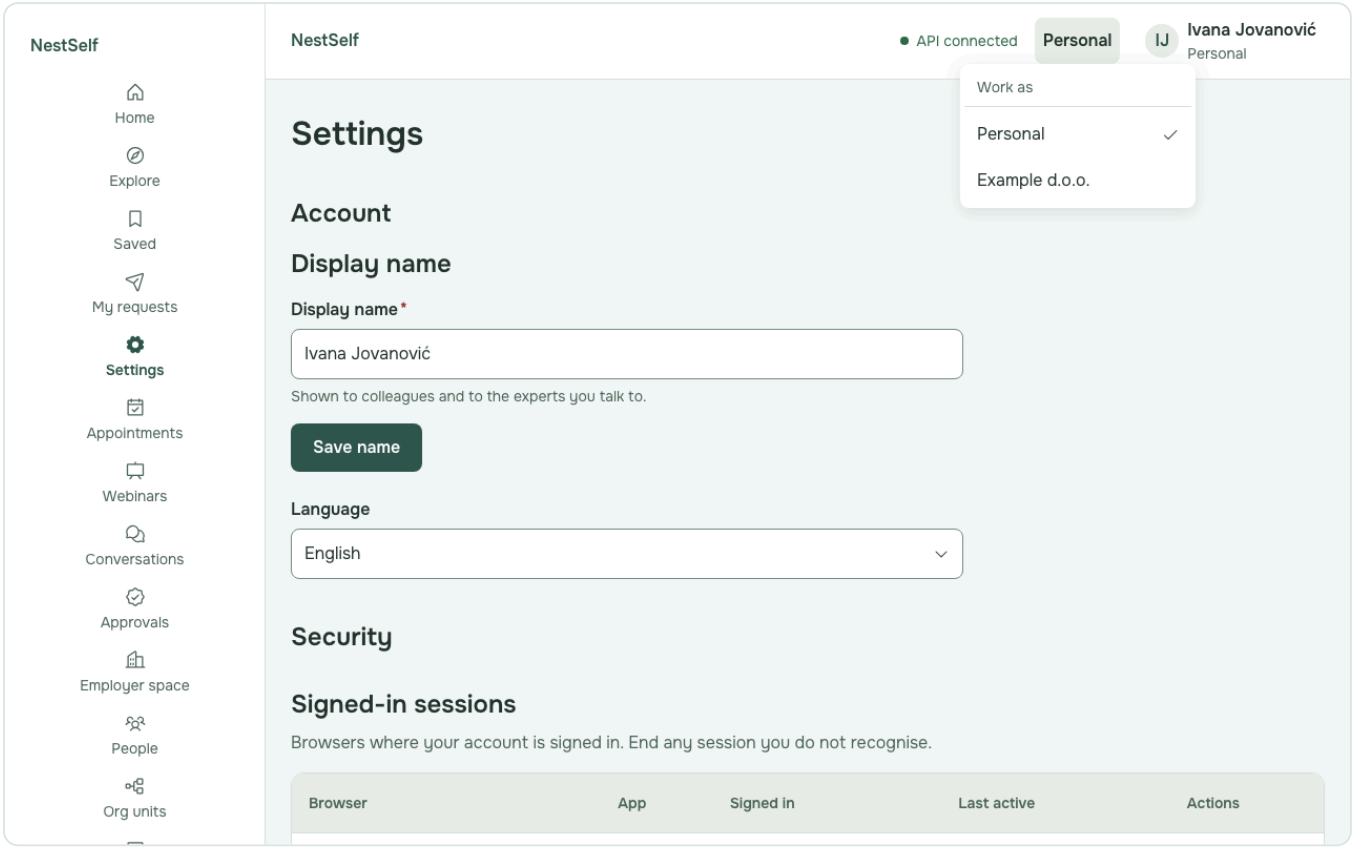
Every auth capability of the API has a screen in the web apps since 2026-10-25 (`implementation.md`, section 3.1, maps each one to its route, gate and API). The screens hide what a role cannot do, but the API decides: hiding a button is a convenience, never the check. Everyone signed in has `/settings` (display name, language, signed-in sessions, sign out everywhere, and the authenticator for password users), the account menu with Sign out, and the context picker in the header when they have more than one context.

Role	What they can do	Where
Owner (`employer_owner`, one per organization)	Grant, revoke and delegate access, preview anyone's effective access, transfer ownership	Workspace `/hr/access`, which opens with the owner template alone (the rest of the HR area needs an HR template; the sample owner is also HR admin)
HR admin (`hr_admin`)	See members, move them between org units and line managers, end memberships (not their own); invite, import and revoke invitations; add, rename and move org units	Workspace `/hr/people` (Members, Invitations) and `/hr/org-units`

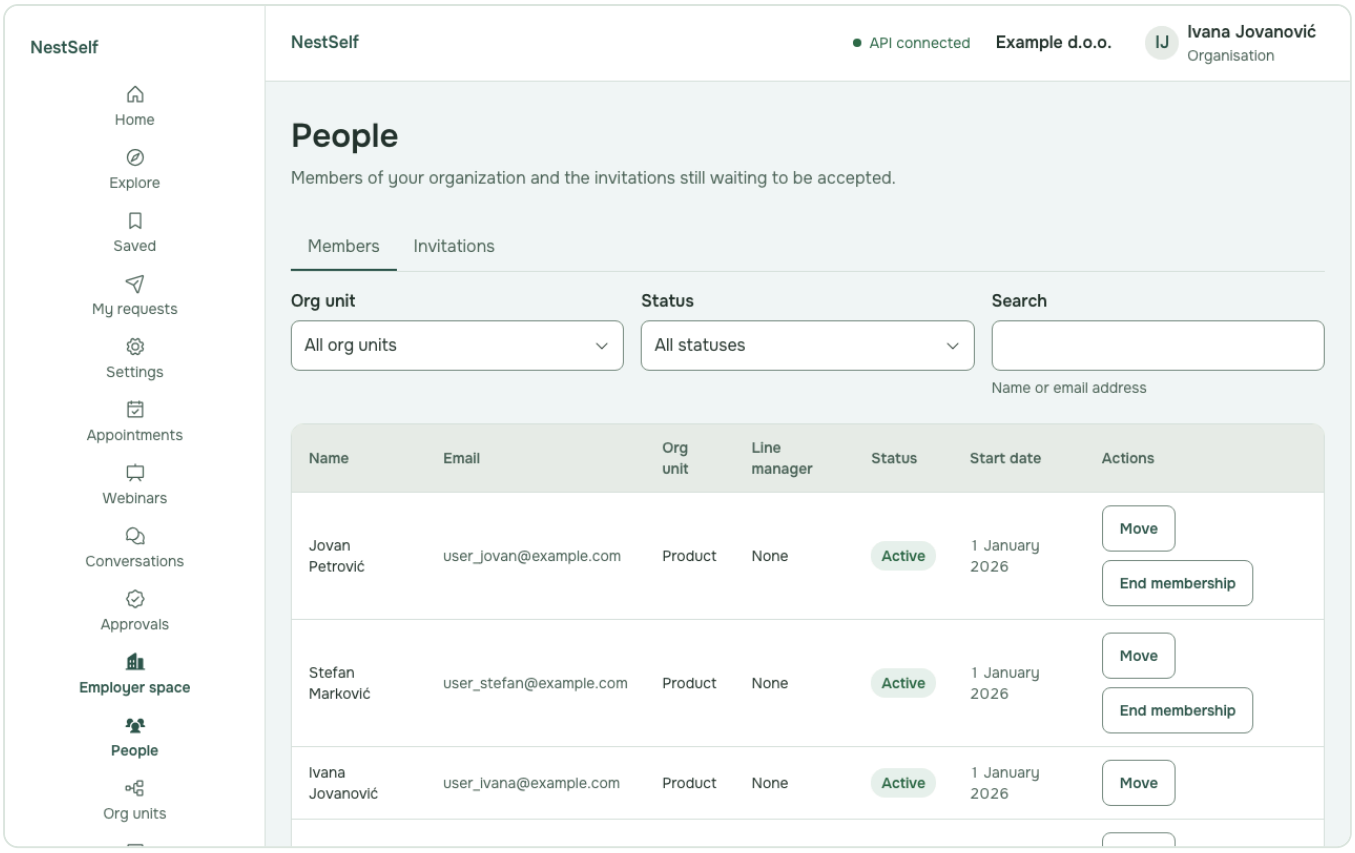
Role	What they can do	Where
Manager (`team_manager`)	Their team's budget, events and approvals; their own account and sessions	Workspace manager area and `/settings`; no access administration
Employee (`employee`)	Their own conversations with an expert, read-only; their account and sessions; switch between personal and organization contexts	Workspace `/conversations` and `/settings`
Expert (`expert`)	Read the conversations of the cases assigned to them (a fresh sign-in or step-up within 15 minutes) and the intake record	Workspace `/cases` (switches them to the expert context)
Staff: platform owner (`platform_owner`)	List organizations, see an organization's members and end memberships, set or transfer its owner, manage platform and organization grants, look up any user's access, suspend and reinstate users, request, approve, deny and revoke emergency access and read a record under it	Admin `/organizations`, `/organizations/:orgId` (Overview, People, Access), `/access`, `/emergency`
Staff: platform owner, SSO (`sso.manage`, held by `platform_owner` only)	Register, verify, test, activate, suspend, edit and reset an employer's SSO connection	Admin `/sso`
Staff: client operations (`client_operations`)	For assigned clients only: members, invitations and imports, end memberships	Admin `/organizations/:orgId/people`, opened by link (the list is platform-wide only)



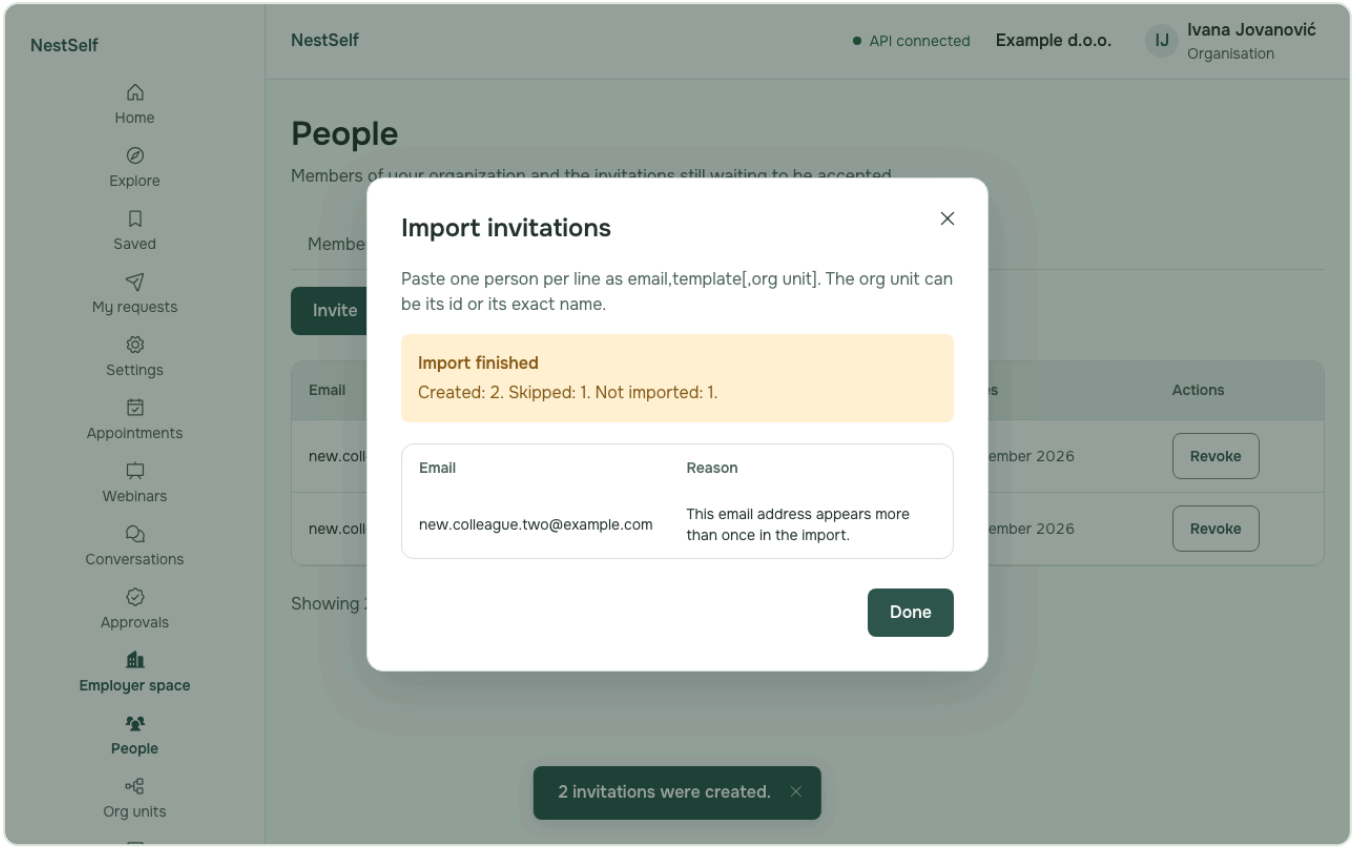
Everyone: the signed-in sessions in `/settings`, with Sign out everywhere.



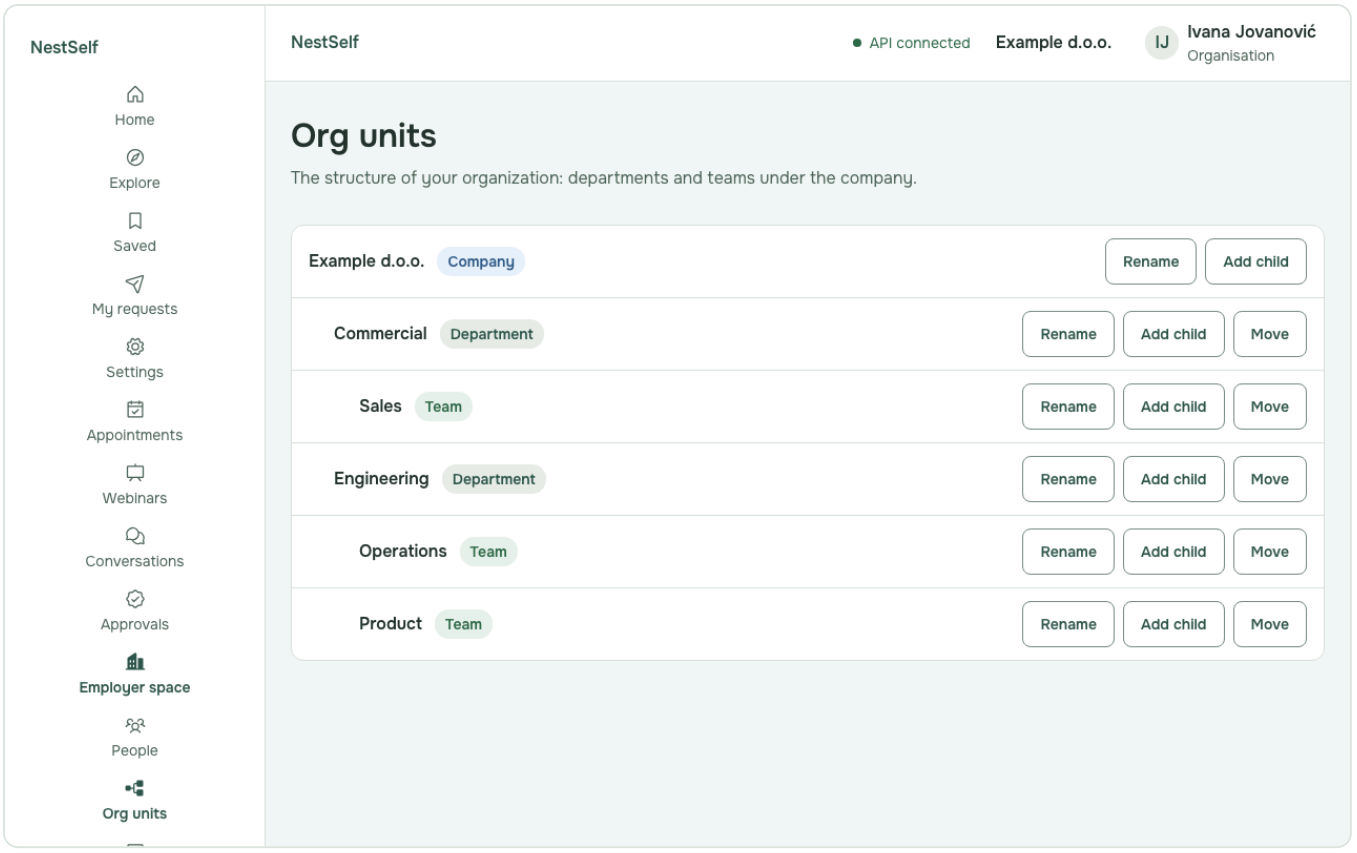
Everyone with more than one context: the picker in the header switches between personal and organization (and expert or operations where held).



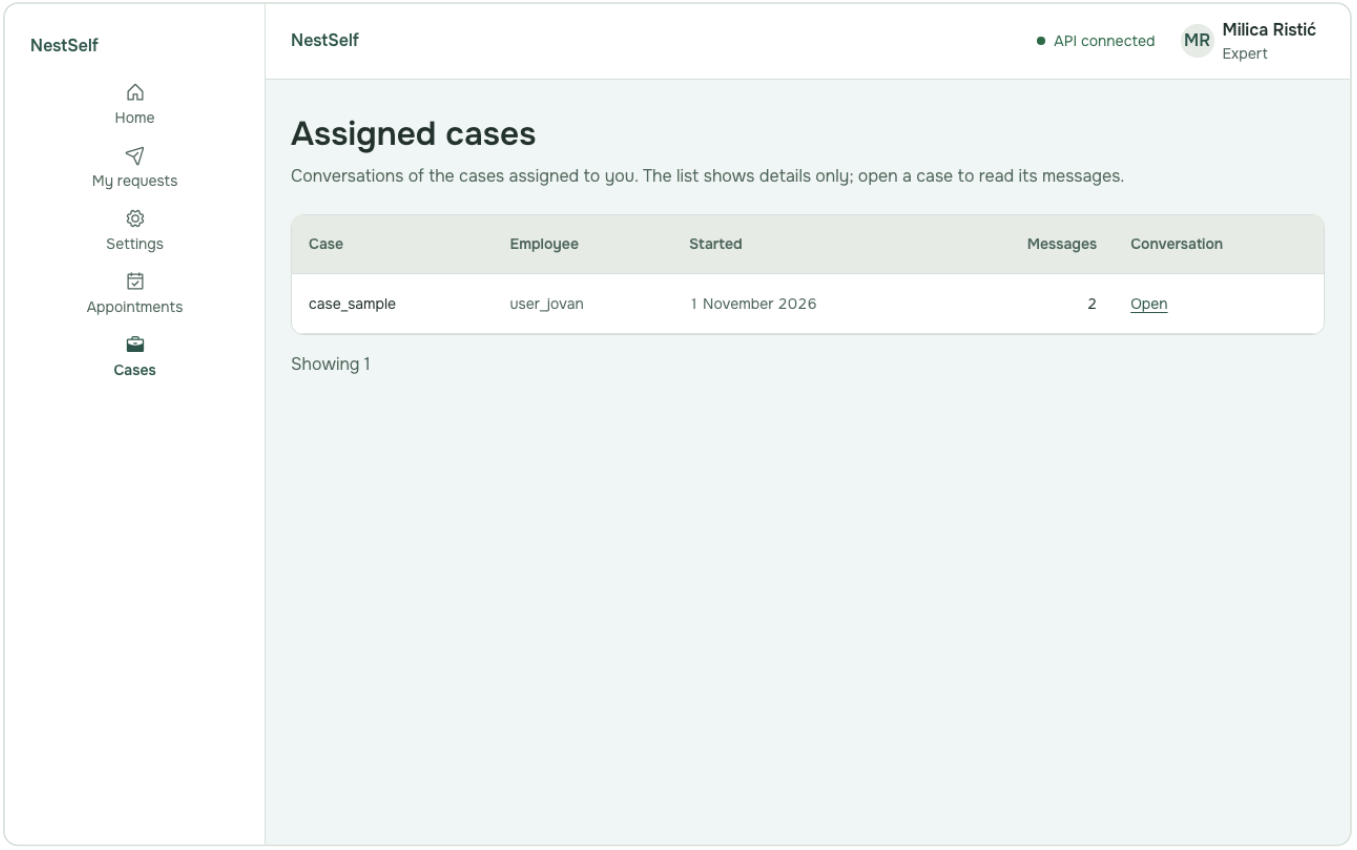
HR admin: `/hr/people`, members with Move and End membership.



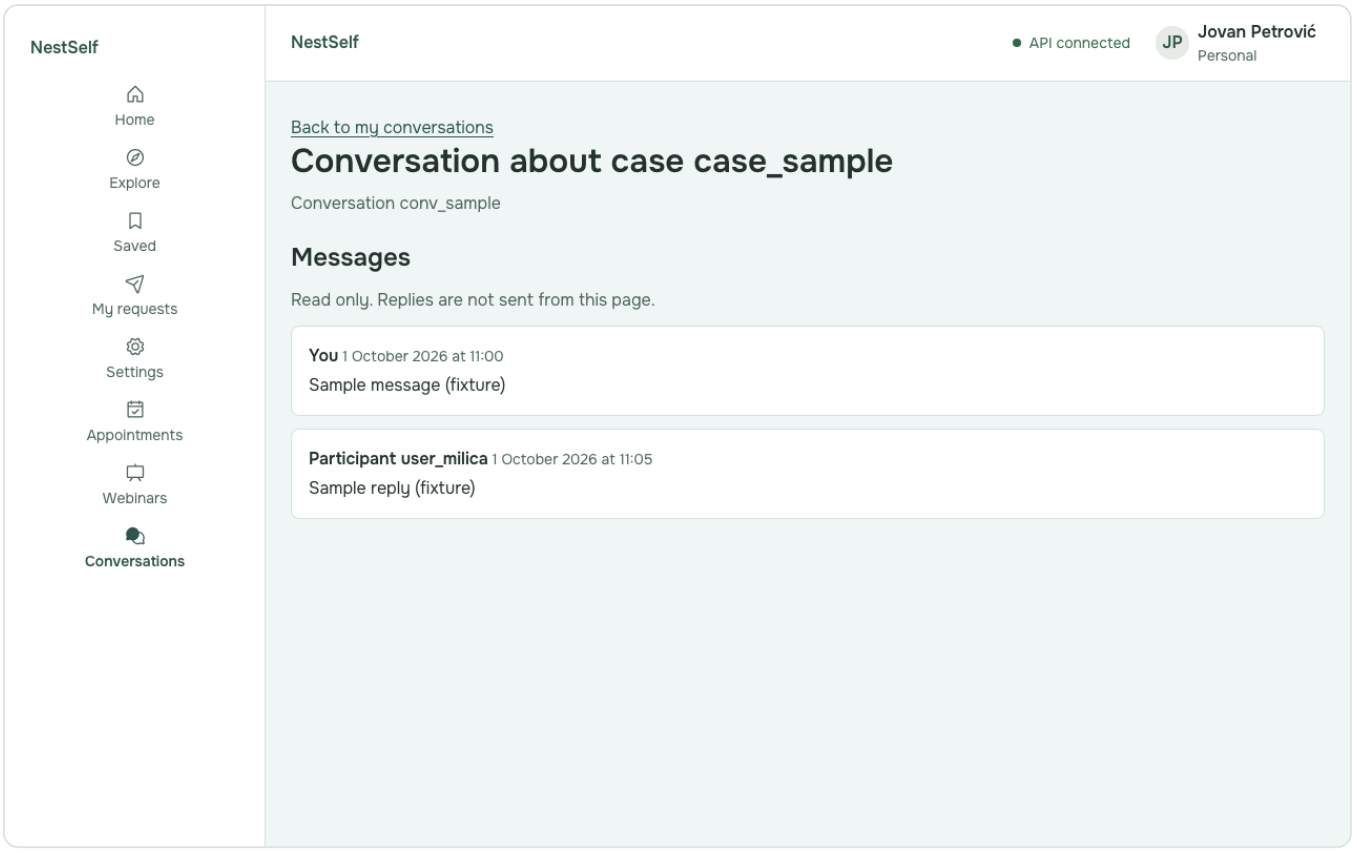
HR admin: importing invitations from pasted lines. Every row lands in one place: created, skipped (already invited or already a member) or not imported, with the reason. The two invitations were created in the disposable e2e database for the screenshot.



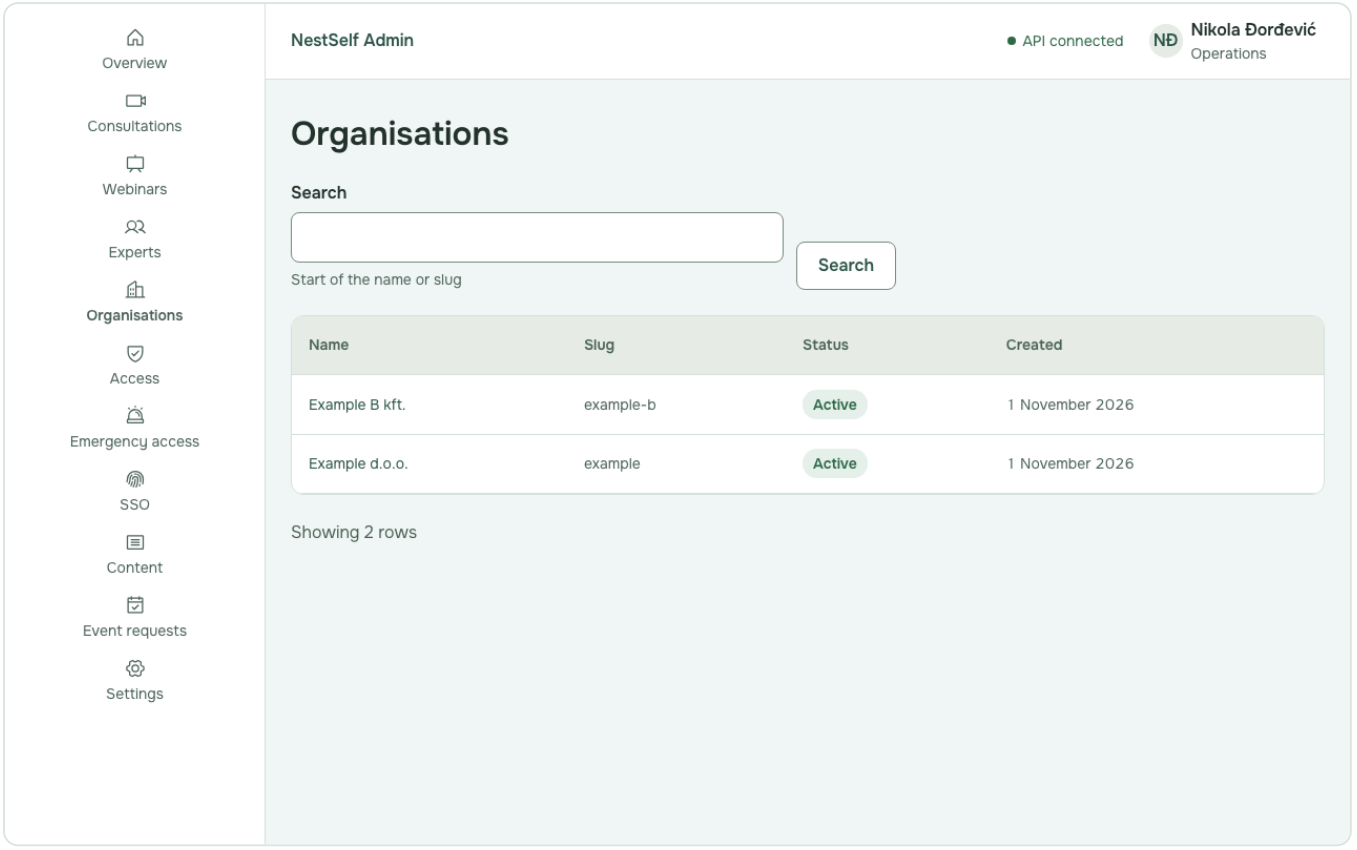
HR admin: the org-unit tree with Rename, Add child and Move.



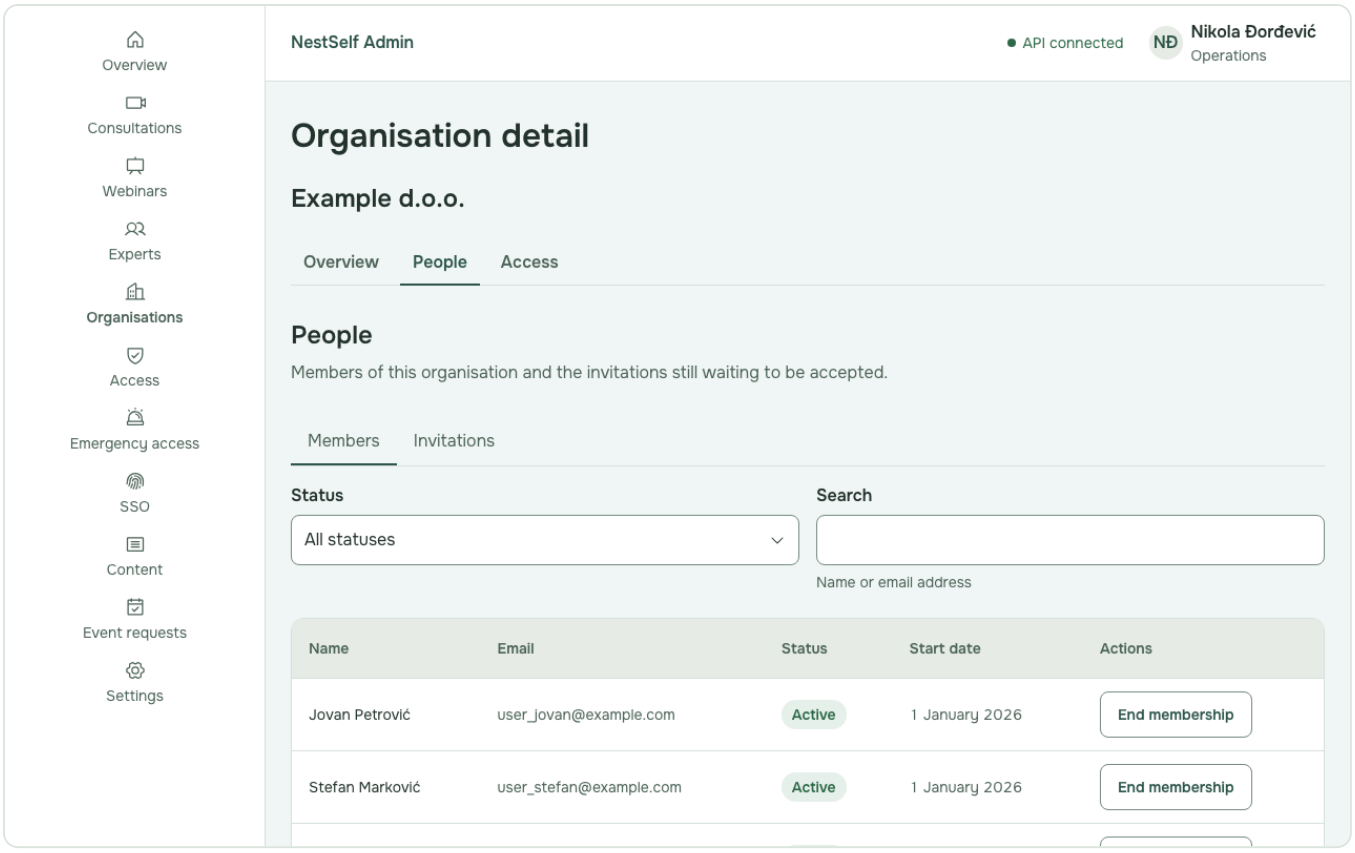
Expert: the conversations of assigned cases.



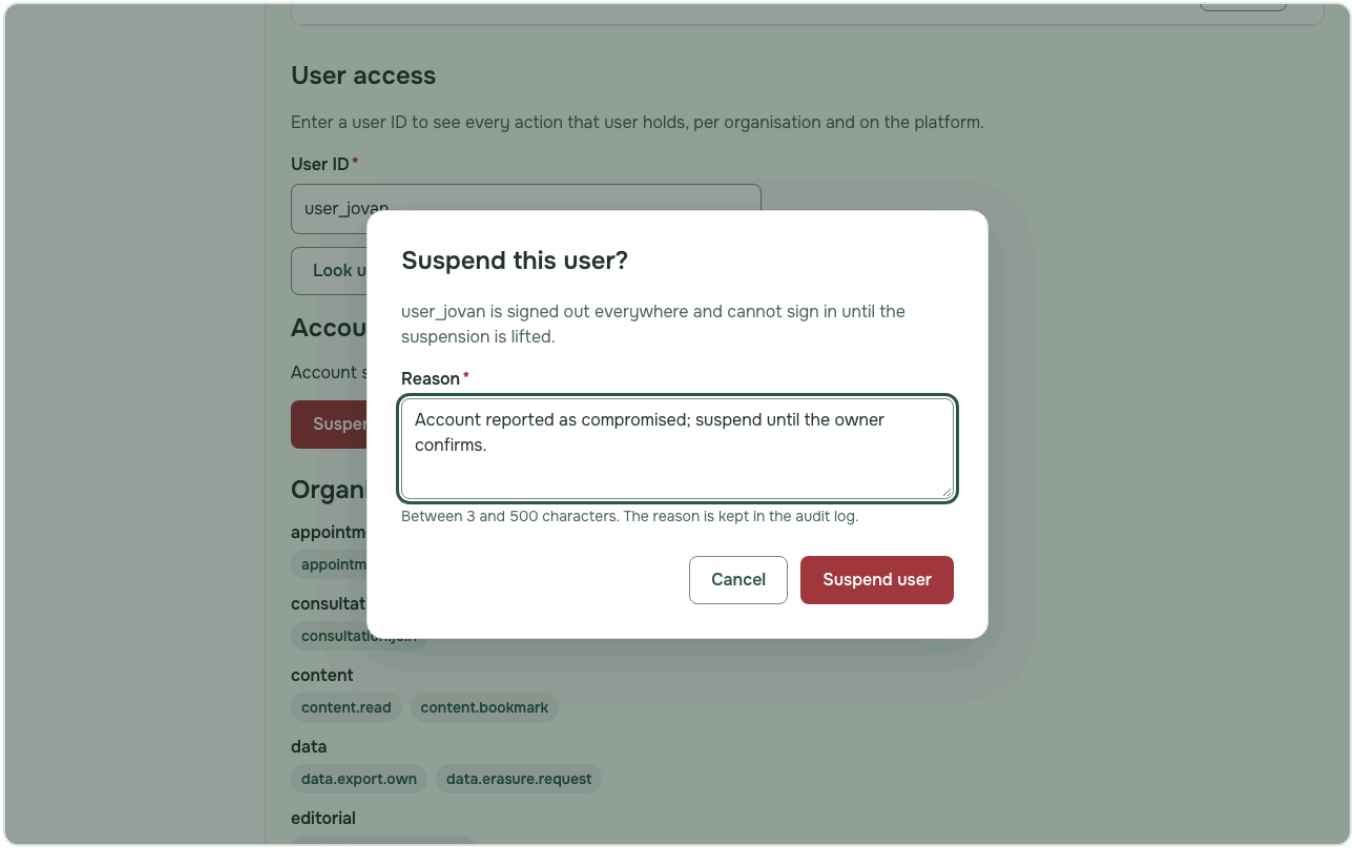
Employee: their own conversation, read-only.



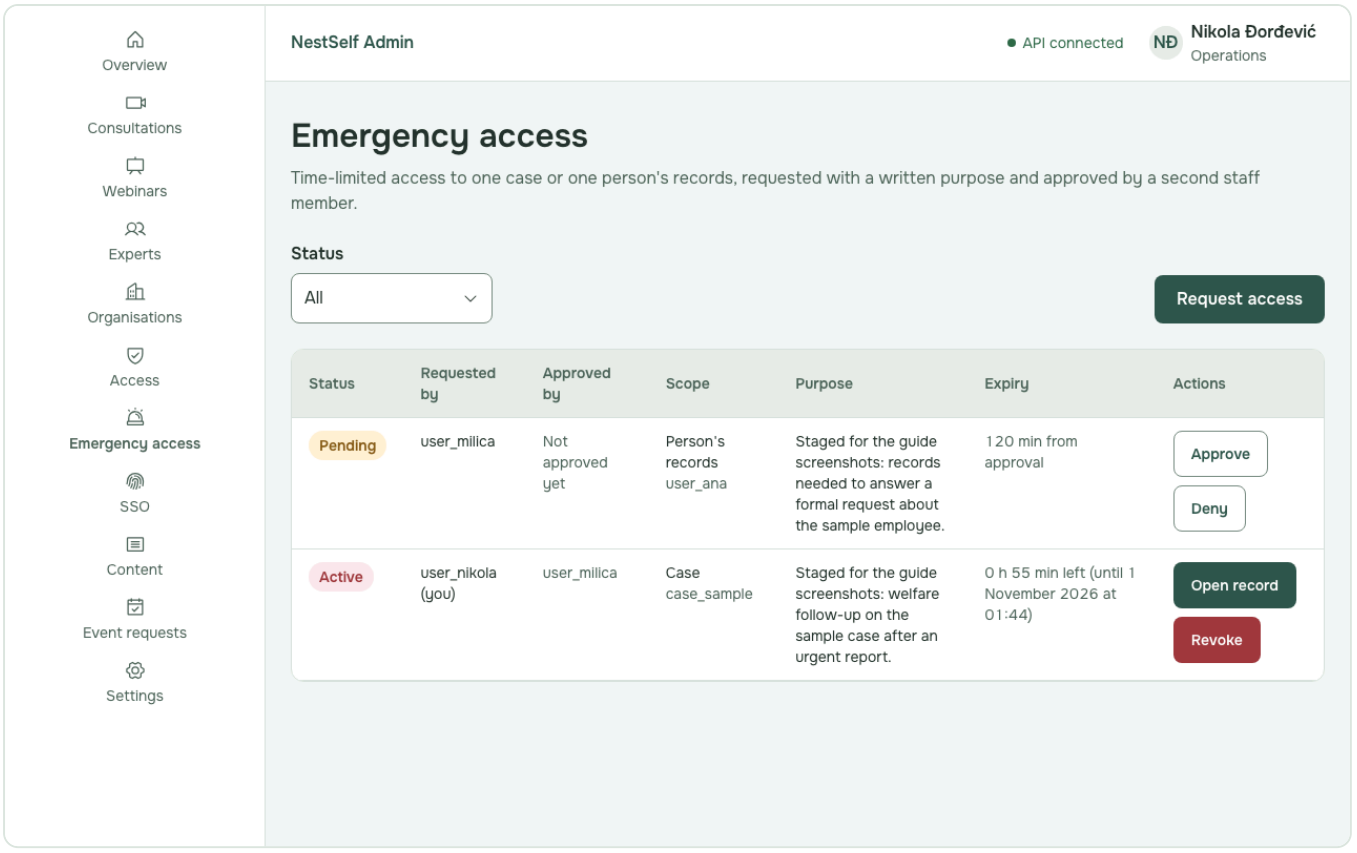
Staff: the organizations list.



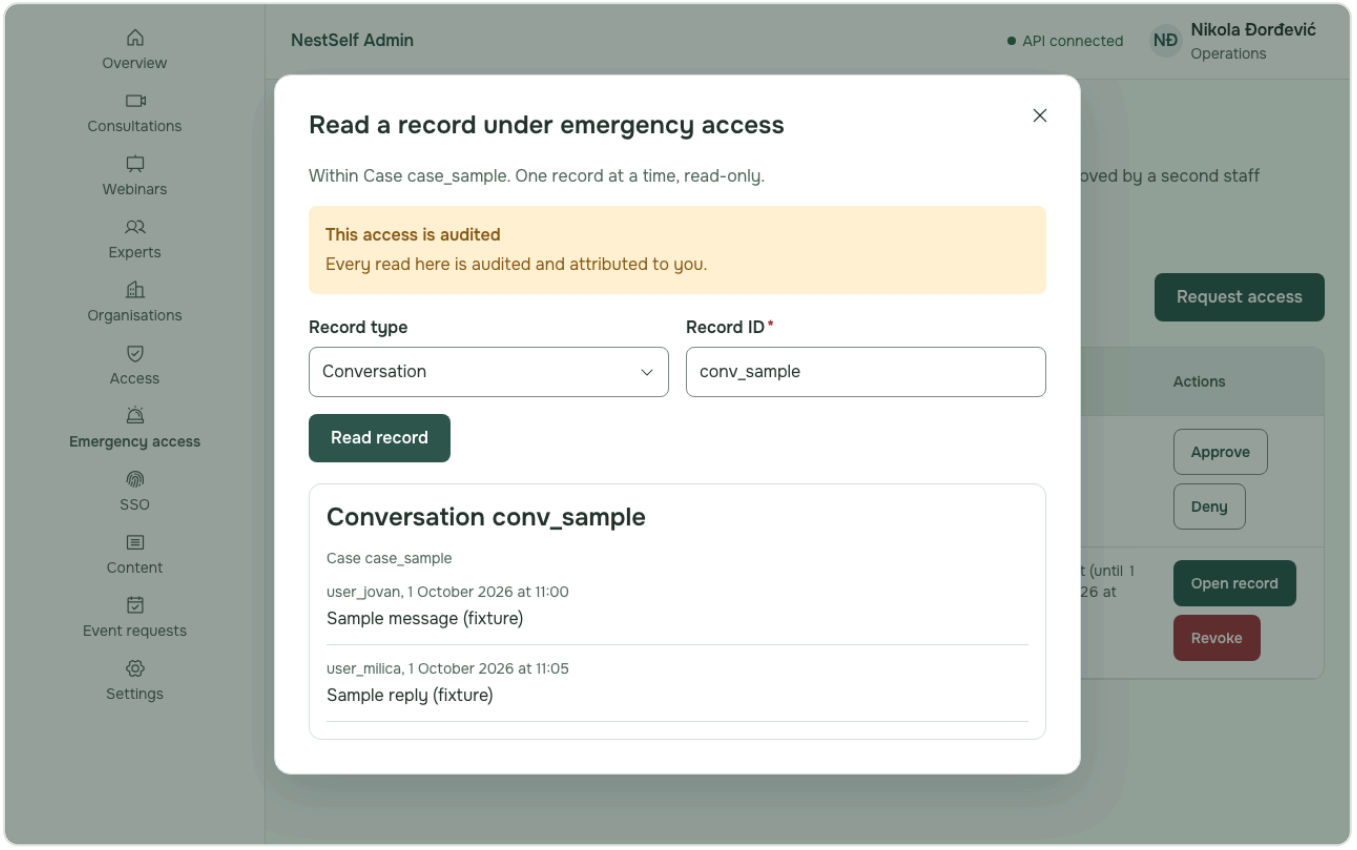
Staff: an organization's members.



Staff: suspending a user from the access lookup asks for a reason; one's own account cannot be suspended.



Staff: emergency access. Approve and Deny appear only on someone else's request; the rows were staged for the screenshot.



Staff: reading a record under approved emergency access, with the audited notice.

Where to look

Question	Place
Which routes exist and which decorator they carry	`apps/api/docs/auth-core.md`
The guards and sign-in services	`apps/api/src/auth/`, `apps/api/src/authz/`
The decision engine and its golden table	`apps/api/src/authz/authorize.ts`, `golden-table.json`
Actions, templates, shared schemas	`packages/contracts/src/`
Database schema, migrations, seed	`apps/api/src/db/schema/`, `apps/api/drizzle/`, `apps/api/src/db/seed/`
Sessions and Redis keys	`apps/api/src/session/`, `apps/api/src/redis/redis-keys.ts`
Sign-in screens	`apps/workspace/src/auth/screens/`, `apps/admin/src/auth/screens/`, `apps/mobile/src/auth/`
Cognito infrastructure and rollout steps	`CDK-setup.md` (repo root), `infra/cdk/lib/auth-stack.ts`, `infra/cdk/README.md`
Nightly jobs and the worker process	`apps/api/src/jobs/`, `apps/api/src/worker/main.ts`
Grant administration, confidential reads, emergency access	`apps/api/src/grants/`, `apps/api/src/professional/`, `apps/api/src/emergency/`
Step-up	`apps/api/src/auth/step-up.service.ts`, `apps/api/src/authz/step-up-freshness.ts`, the web apps' `src/auth/step-up-dialog.tsx`
Access screens	`apps/workspace/src/features/access/`, `apps/admin/src/features/access/`
Settings, people, org units, organizations, emergency, expert screens	`apps/{workspace,admin}/src/features/` (`account`, `context`, `people`, `org-units`, `organizations`, `users`, `emergency`, `cases`); the full map is `implementation.md` section 3.1
Browser journeys and screenshots	`e2e/auth/`, `e2e/screenshots/`, `e2e/README.md`
How each role uses the product, step by step	`docs/auth/guides/README.md` (one tutorial PDF per role)
Enterprise SSO	`apps/api/src/sso/`, `apps/api/src/auth/sso-callback.service.ts`, `federated-binding.service.ts`, `sso-bind.service.ts`, `discover.service.ts`, `apps/api/src/identity/cognito-sso-ops.ts`, `apps/admin/src/features/sso/`, `implementation.md` section 22
Email and the codes NestSelf sends	`apps/api/src/mailer/` (the shared mailer, templates under `templates/`), `apps/api/src/auth/nestself-otp.service.ts`, `implementation.md` sections 6 and 24

Question	Place
Phase 2 to 4 status and what is left	`docs/tasks/auth/phase-2-readiness.md`, `implementation.md` section 21
Environment variables	`docs/environment.md`, `apps/api/src/config/env.ts`

Running it locally

```
pnpm install && pnpm db:up
cp apps/api/.env.example apps/api/.env
pnpm --filter @nestself/api db:migrate && pnpm --filter @nestself/api db:seed
pnpm --filter @nestself/api dev
pnpm --filter @nestself/api worker:dev   # optional: jobs, needs REDIS_URL (runs through the
Nest compiler)
```

Stub mode uses a fake identity provider pre-loaded with the sample people. Employees (`user_jovan@example.com`, `user_ivana@example.com`, `user_stefan@example.com`, `user_dragan@example.com`) sign in with the code printed in the API log, and Ivana reaches `/hr/access`. The codes NestSelf mails itself (SSO users' step-up, the confirm-email step) are never logged: set `FAKE_IDP_FIXED_OTP=123456` to use those flows locally; staff and experts (`user_nikola@example.com`, `user_milica@example.com`, and the editorial staff `user_katarina@example.com`, `user_darko@example.com`, `user_teodora@example.com`, `user_vuk@example.com`) use the password `Nestself-dev-1234!` and enrol an authenticator on first sign-in. Set `VITE_DATA_SOURCE=api` in a web app's `.env.local` to use the real screens. Skip `pnpm db:up` when PostgreSQL and Redis already run outside Docker; the Compose file claims port 6379 and fails against a local Redis or Valkey.

A real Cognito dev pool exists since 2026-10-24 (`eu-central-1_g6OGYONQ0` in account `634972095709`). Set `AUTH_MODE=cognito` with the values `infra/cdk/scripts/env-from-outputs.mjs dev` prints, and note that pool usernames are email addresses, not handles. `implementation.md` section 17.1 has the deployed coordinates and the three traps worth knowing before touching it.

Glossary

- **Principal**: the resolved identity for a request (user, memberships, active context, how they authenticated).
- **Context**: the hat the user wears right now: personal, a specific organization, expert, or operations.
- **Grant**: a user's permission to perform actions within a scope, with conditions and a validity window.
- **Template**: a named starting set of actions (`hr_admin`, `team_manager`, `employee`, ...) grants are created from.
- **Assignment**: a link to a work item (case, queue, client) that confers access without a grant.
- **Closure table**: every ancestor to descendant pair of the org tree, so scope checks are a set lookup.
- **Step-up**: re-authenticating recently before a sensitive action; enforced by the API.
- **Delegation**: handing part of one's own grant to another member, at most three levels deep and never beyond the parent grant.

What is not done yet

Auth phase 4 (2026-10-31) closed what would have stopped the first employer on SSO and the auth debts that needed no decision: per-network throttles, step-up for SSO users, the confirm-email step, SSO on mobile, certificate dates and alerts for metadata URLs, the custom Cognito domain in CDK, audit rows in the same transaction, coded import results, the owner's access page, member search, the "could not load your account" state and confidential reads that no longer reveal whether a record exists. Still open, each for a reason (`implementation.md` section 21 has the full list):

- **Waiting for the pilot organisation**: the live test with its identity provider, the decision on its OIDC email claim, a sweep of sign-ins that were never confirmed, retries for large invitation imports, throttle budgets checked against real traffic, and member search in the admin app's pickers.
- **Waiting for AWS**: deploying staging and production with the custom auth domain (before any employer configures its IdP), confirming that the load balancer answers CloudFront only, a check that the dev auth stack is unchanged, a secret origin header, a marketing distribution, the security baseline, the weekly Cognito export and the audit export.
- **Owner decisions**: the authenticator for emergency step-up, the presenter template, retention periods, who approves emergency access, the staging date, IdP-initiated sign-in and SCIM, and the compliance track (privacy notices, processing agreement, breach procedure), which must land before the first real employee signs in.
- **Not planned yet**: mobile step-up and administration screens, screens for approval authorities and grant conditions, file and export authorization, retiring the community feature, and smaller items.

Who administers grants inside an organization was decided on 2026-10-24: the single employer owner, and NestSelf staff. The status per task is `docs/tasks/auth/phase-2-readiness.md`.